Department Artificial Intelligence and Data Science

Database Management System (PCC-251- AID)

Unit – 2

Smart Data Querying with SQL & PL/SQL

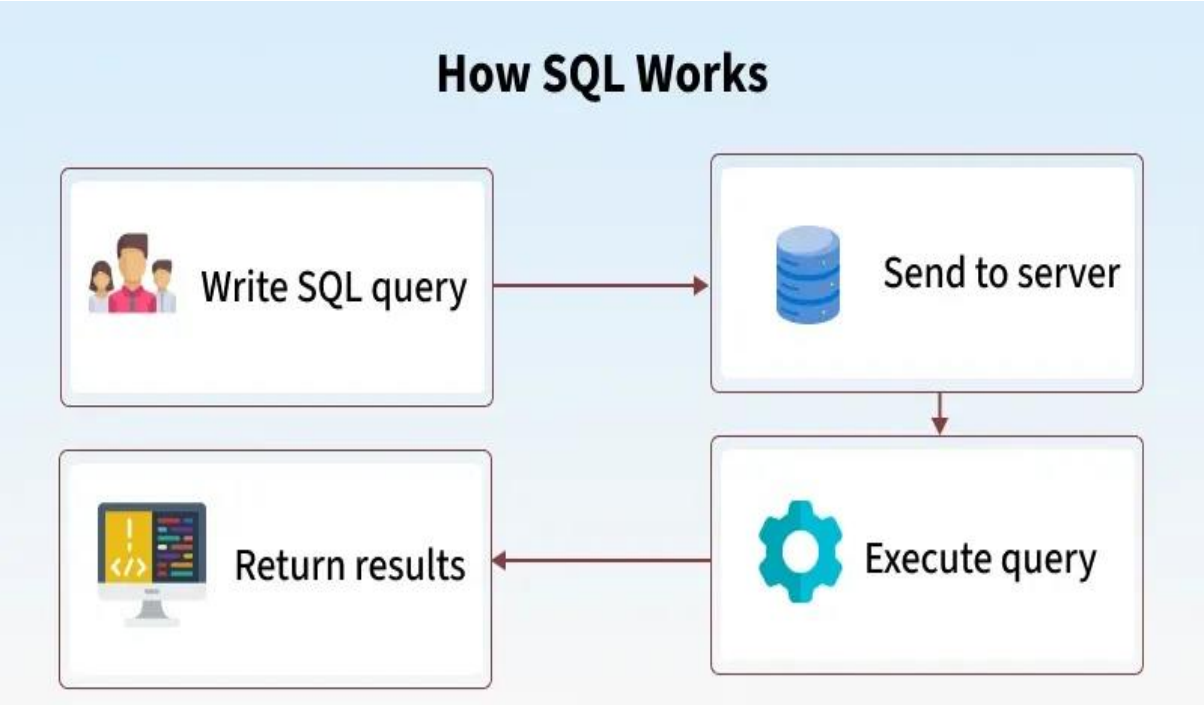
SQL (Structured Query Language)-

Structured Query Language (SQL) is the standard language used to interact with relational databases.

- It allows users to store, retrieve, update and manage data efficiently through simple commands.
- It is known for its user-friendly syntax and powerful capabilities, SQL is widely used across industries.

SQL Working

We interact with databases using SQL queries. DBMS tools like MySQL and SQL Server have their own SQL engine and an interface where users can write and execute SQL queries.



Below are the detailed steps involved in the SQL query execution.

- **Input** – The user sends a SQL query (like SELECT or INSERT).
- **Parsing** – The system checks if the query is written correctly.
- **Optimization** – It chooses the fastest way to run the query.
- **Execution** – The database runs the query.
- **Output** – The result or confirmation is sent back to the user.

Components of a SQL System

- **Database:** A structured collection of data stored in tables with rows and columns.
- **Tables:** Store data and apply rules to keep it accurate and consistent.
- **Indexes:** Help the database find data faster without scanning the whole table.
- **Views:** Virtual tables created from SELECT queries for easy access to data.
- **Stored Procedures:** Pre-saved SQL code that runs tasks and improves performance and security.
- **Transactions:** Group of SQL actions that either all succeed or all fail to keep data safe.
- **Security & Permissions:** Control who can view, change, or manage database data.
- **Joins:** Combine data from different tables based on relationships.

Rules for Writing SQL Queries

There are certain rules for SQL which would ensure consistency and functionality across databases. By following these rules, queries will be well formed and well executed in any database.

- **Semicolon (;):** Ends an SQL statement.
- **Case-Insensitive:** SQL keywords like SELECT and INSERT are not case-sensitive.
- **Whitespace:** Spaces and new lines are allowed for better readability.
- **Reserved Words:** Do not use SQL keywords as names, or write them in quotes/backticks.

Comments

- Single-line: -- comment
- Multi-line: /* comment */

Data Constraints: Use NOT NULL, UNIQUE, PRIMARY KEY, etc., to ensure data accuracy.

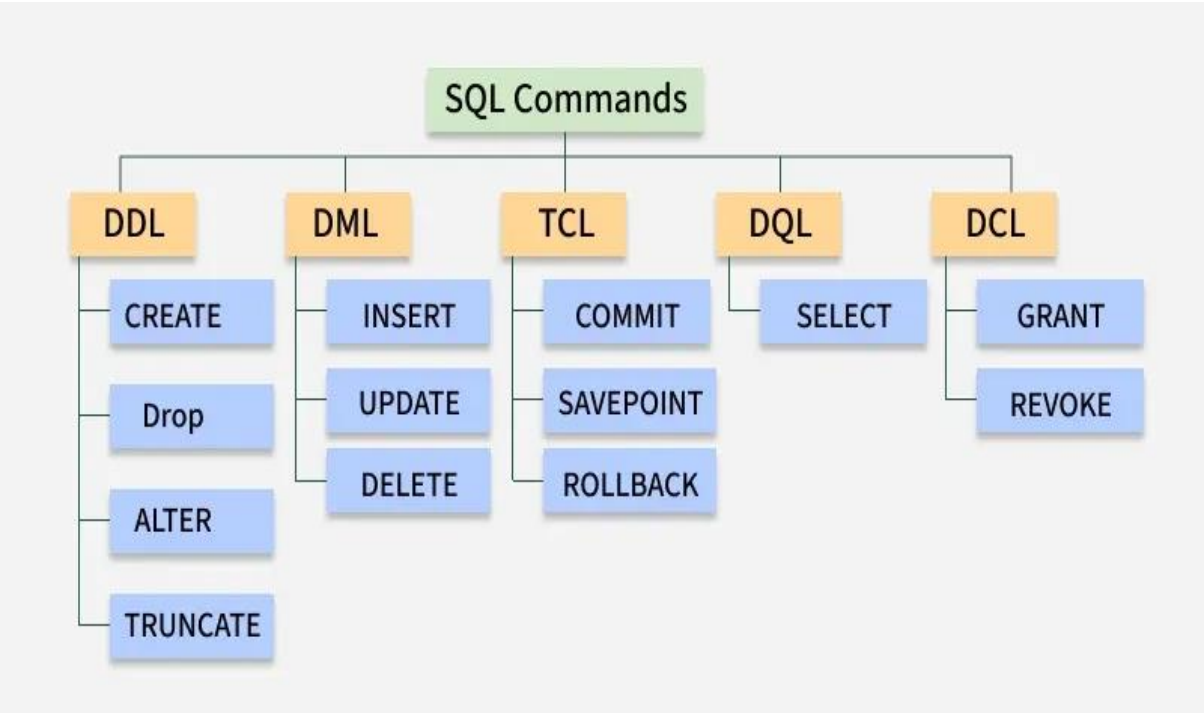
String Values: Enclose strings in single quotes ('text').

Naming Rules:

- We Start with a letter
- We can only use Max 30 characters
- We only use letters, numbers and underscores (_)

Different SQL Commands or Queries

Structured Query Language (SQL) commands are standardized instructions used by developers to interact with data stored in **relational databases**. These commands allow for the creation, manipulation, retrieval and control of data, as well as database structures. SQL commands are categorized based on their specific functionalities:



1. Data Definition Language(DDL)

DDL (Data Definition Language) commands are used to create, change, and delete database objects. Database engineers use commands like CREATE to make tables, views, and indexes, and ALTER or DROP to modify or remove them.

Command	Description
CREATE	Creates a new table, a view on a table, or some other object in the database.
ALTER	Modifies an existing database object, such as a table
DROP	Deletes an entire table, a view of a table, or other objects in the database
TRUNCATE	Removes all records from a table but keeps the table structure intact.
RENAME	It is used to change the name of an existing database object, such as a table.

SQL CREATE TABLE

The CREATE TABLE statement in SQL is used to define a new table in a database. It specifies the table name, column names, and their data types. This command lays the foundation for storing and organizing data.

Syntax:

```
CREATE table table_name
(
```

```
Column1 datatype (size),
column2 datatype (size),
.
.
columnN datatype(size)
);
```

- **table_name**: The name you assign to the new table.
- **column1, column2, ...** : The names of the columns in the table.
- **datatype(size):** Defines the data type and size of each column.

Example: Create a Customer Table

Let's walk through a practical example where we create a Customer table that stores customer data. We will define various columns such as CustomerID, CustomerName, Country, Age, and Phone with appropriate data types and constraints.

Query:

```
CREATE TABLE Customer(
    CustomerID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    Country VARCHAR(50),
    Age INT CHECK (Age >= 0 AND Age <= 99),
    Phone int(10)
);
```

Output:

CustomerID	FirstName	LastName	Country	Age	Phone
enmpty					

Explanation:

- CustomerID is an integer and serves as the PRIMARY KEY, ensuring each record is unique.
- FirstName, LastName, and Country are VARCHAR fields to store variable-length text.
- Age has a CHECK constraint, ensuring it's within the range of 0 to 99.
- Phone is an integer field, although in real scenarios, a VARCHAR would often be used for storing phone numbers to allow for leading zeros and formatting.

ALTER TABLE Examples

- Add Email Column

```
ALTER TABLE Customer
ADD Email VARCHAR(100);
```

Modify Age Column

```
ALTER TABLE Customer
MODIFY Age NUMBER(3);
```

Drop Phone Column

```
ALTER TABLE Customer
DROP COLUMN Phone;
```

TRUNCATE Example

TRUNCATE TABLE Customer;

- Deletes all records
- Keeps table structure

DROP Example

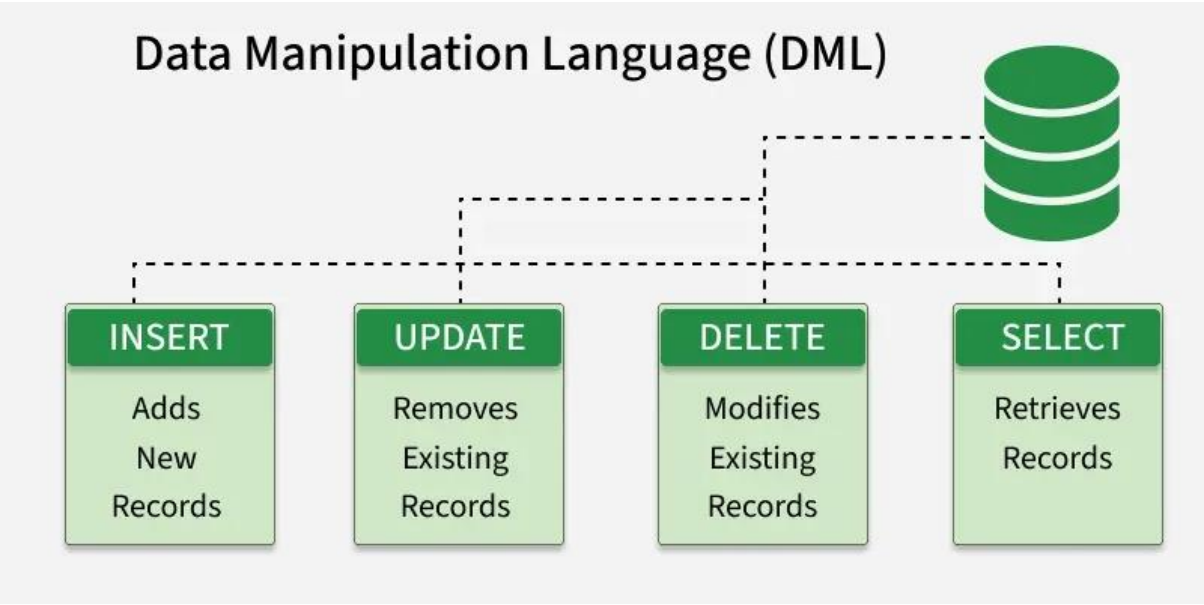
DROP TABLE Customer;

- Deletes entire table permanently
-

2. Data Manipulation Language (DML)

DML stands for Data Manipulation Language. Tables and formulas are helpful when communicating with data stored up to a point in a database through SQL (Structured Query Language), but a time comes when we actually want to execute some fairly complicated data interactions. DML is a way to inform a database precisely what we want it to do by conversing in a manner that it has been built to comprehend from scratch.

- DML provides an easy way to work with existing data, whether you want to add, update, move, or delete it.
- Every change made by a DML statement is treated as a transaction, so it requires transaction control.
- DML is a subset of SQL that modifies the actual data stored in tables.
- It focuses on database performance and works efficiently with systems like HDFS that follow an append-only storage model.



Generally, DML commands fall into one of four primary categories:

- **INSERT** adds fresh data to a table.
- **UPDATE** Change the data that is already in a table.
- **DELETE** takes a record out of a table.
- **SELECT** Get information out of one or more tables.

Characteristics of DML

It performs interpret-only data queries. It is used in a database schema to recall and manipulate the information. DML It is a dialect which is used to select, insert, delete and update data in a database. Data Manipulation Language (DML) commands are as follows:

SELECT Command

This command is used to get data out of the database. It helps users of the database to access from an operating system, the significant data they need. It sends a track result set from one tables or more.

Syntax :

```
SELECT *
FROM <table_name>;
```

Example:

```
SELECT *
FROM students;

OR
```

```
SELECT *
FROM students
where due_fees <=20000;
```

INSERT Command

This command is used to enter the information or values into a row. We can connect one or more records to a single table within a repository using this instruction. This is often used to connect an unused tag to the documents.

Syntax:

```
INSERT INTO <table_name> ('column_name1' <datatype>,
'column_name2' <datatype>)
VALUES ('value1', 'value2');
```

Example :

```
INSERT INTO students ('stu_id' int, 'stu_name' varchar(20),
'city' varchar(20))
VALUES ('1', 'Nirmit', 'Gorakhpur');
```

UPDATE Command

This command is used to alter existing table records. Within a table, it modifies data from one or more records. This command is used to alter the data which is already present in a table.

Syntax:

```
UPDATE <table_name>
SET <column_name = value>
WHERE condition;
```

Example:

```
UPDATE students
SET due_fees = 20000
WHERE stu_name = 'Mini';
```

DELETE Command

It deletes all archives from a table. This command is used to erase some or all of the previous table's records. If we do not specify the 'WHERE' condition then all the rows would be erased or deleted.

Syntax:

```
DELETE FROM <table_name>
WHERE <condition>;
```

Example:

```
DELETE FROM students
WHERE stu_id = '001';
```

Benefits of DML

- DML statements could alter the data that is contained or stored in the database.
- It delivers effective human contact with the machine.
- User could specify what data is required.
- DML aims to have many different varieties and functionalities between vendors providing databases.

Limitations of DML

- We cannot use DML to change the structure of the database.
- Limit table view i.e., it could conceal some columns in tables.
- Access the data without having the data stored in the object.
- Unable to build or erase lists or sections using DML.

3. Data Query Language(DQL)

Data retrieval instructions are written in the data query language (DQL), which is used to access relational databases. The SELECT command is used by software programs to filter and return particular results from a SQL table.

Data Query Language (DQL) is a component of **Structured Query Language (SQL)** focused on retrieving data from databases using the **SELECT** statement.
It allows users to extract specific information based on defined criteria, enabling efficient data analysis and decision-making. In this article, We will learn about the **DQL Full Form** in detail by understanding various aspects in detail.

Introduction to Data Query Language (DQL)

- **Structured Query Language (SQL)** is a **nonprocedural** language used for retrieving data from queries. It was introduced by **IBM** as a part of the **R** project. It was declared as a standard language by **ANSI** and ISO. **DQL** statements are used for performing queries on the data within schema objects.
- The purpose of the DQL Command is to get some schema relation based on the query passed to it.
- We can define DQL as follows it is a component of SQL statement that allows getting data from the database and imposing order upon it.
- It includes the SELECT statement. This command allows getting the data out of the database to perform operations with it.
- When a **SELECT** is fired against a table or tables the result is compiled into a further temporary table, which is displayed or perhaps received by the program i.e. a front-end. **Data Query Language (DQL)** or **Data Retrieval Language (DRL)**.

Data Query Language has commands that retrieve the data from the query. It has a single command: select. It has subcategories:

- DDL or Data Definition Language has commands like create, rename, and alter.
- DML or Data Manipulation Language has commands like update, insert, and delete.
- DCL or Data Control Language has commands like grant and revoke.
- TCL or Transaction Control Language has commands like rollback, and commit.

Characteristics of Data Query Language (DQL)

It is used for retrieving data. A user can retrieve data according to his/her requirements using the select command.

1. **Data Retrieval**: DQL is used only to retrieve data.
2. **Flexibility in query**: The SELECT statement enables a user to obtain specific data based on criteria or conditions.
3. **Data Ordering**: It keeps the results sorted and filtered. This way, work on database data becomes even more structured and organized.

Example:

If a user wants to retrieve all the from the employee table, the query must be given as follows:

```
select * from employee;
```

asterisk mark (*) indicates that all the rows are retrieved from the table.

If a user wants to know the average salary of the employees, the query must be given as follows:

```
select avg(salary)
from employee;
```

This displays the average salary of the employees. Here asterisk mark isn't needed as we need to retrieve only particular rows.

If a user wants to know the name of the employee whose salary is less than 10, 000; the query is:

```
select ename
from employee
where salary<=10,000;
```

A condition is placed to retrieve the data and a relational operator is used as well.

Advantages of DQL

- SELECT is a standard command used for almost every query.
- It can also be written as SELECT as SQL is a case-insensitive language.
- It makes the data retrieving process easier.
- No coding is needed.
- This domain language can be used for communicating with the databases and receive answers to complex questions in seconds.

Disadvantages of DQL

- Data Query Language has no disadvantages.
- It is not possible to imagine Structured Query Language without the select command.
- Interfacing a DQL database is more complex than adding a few lines of code.
- The operating cost of some DQL versions makes it difficult for some programmers to access them.

Applications of DQL

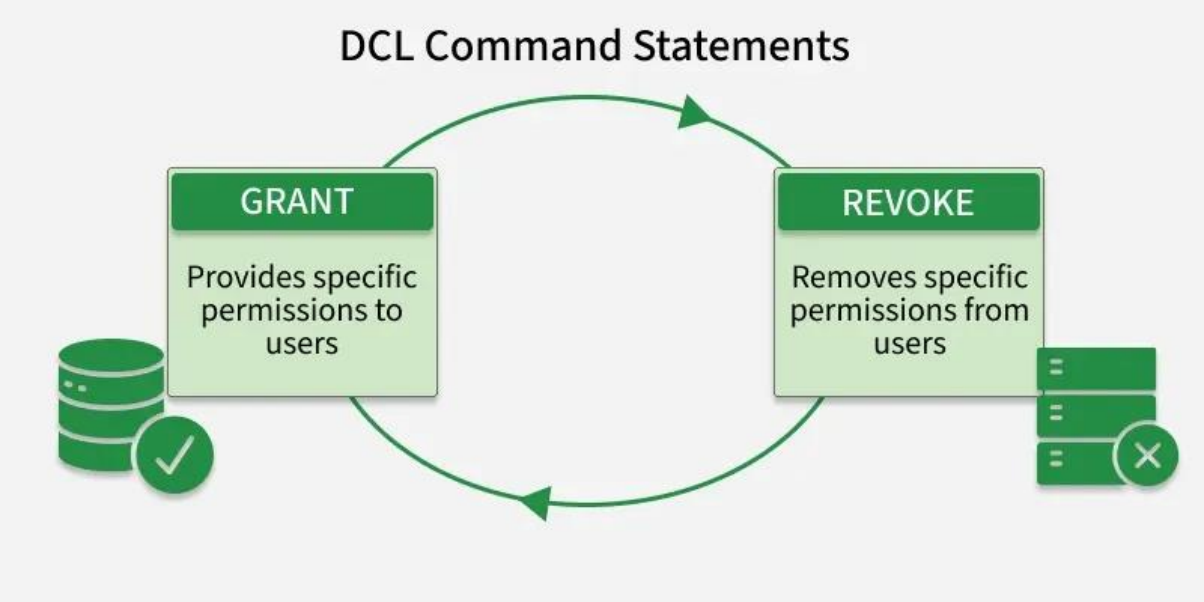
- Analytical Queries.
- Retrieve Information from the database.
- Modify the index structures and database table.
- Such questions are analytical in nature and DQL is used abundantly to run analytical queries in retrieving worthwhile knowledge from data.
- **Database Information Retrieval**: This helps in retrieving a certain information from the database for achieving a task, such as employee's details or sales records, etc.
- **Modify Index Structures**: DQL creates modifications in database structures. To enable improved queries, indexing is also done.

4. Data Control language(DCL)

DCL commands manage user access to the database by granting or revoking permissions. Database administrators use DCL to enforce security and control access to database objects.

Command	Description
GRANT	Gives a privilege to the user.
REVOKE	Takes back privileges granted by the user.

The full form of DCL is Data Control Language in Structured Query Language (SQL). DCL commands are used to control privileges in the database. The privileges (right to access the data) are required for performing all the database operations, like creating tables, views, or sequences. DCL statements are used to perform the work related to the rights, permissions, and other controls of the database system.



There are two types of **privileges** in the database:

- **System Privilege**
- **Object Privilege**

Need Of DCL commands

- Unauthorized access to the data should be prevented in order to achieve security in our database
- DCL commands maintain the database effectively than anyone else other than database administrator is not allowed to access the data without permission.
- These commands provide flexibility to the data administrator to set and remove database permissions in granular fashion.

Commands in DCL

The two most important **DCL commands** are **GRANT** and **REVOKE.**

GRANT

This command is used to grant permission to the user to perform a particular operation on a particular object. If we are a **database administrator** and we want to restrict user accessibility such as one who only views the data or may only update the data.
we can give the privilege permission to the users according to our wish.

Syntax

```
GRANT privilege_list

ON Object_name

TO user_name;
```

REVOKE

This command is used to take permission/access back from the user. If we want to return permission from the database that we have granted to the users at that time we need to run REVOKE command.

Syntax

```
REVOKE privilege_list

ON object_name

FROM user_name;
```

Following commands are granted to the user as a Privilege List:

- **EXECUTE**
- **UPDATE**
- **SELECT**
- **DELETE**
- **ALTER**
- **ALL**

Advantages of DCL commands:

- It allows to restrict the user from accessing data in database.
- It ensures security in database when the data is exposed to multiple users.
- It is the wholesome responsibility of the data owner or data administrator to maintain the authority of grant and revoke privileges to the users preventing any threat to data.

- It prevents other users to make changes in database who have no access to Database

Disadvantages of DCL commands:

- DCL targets only restriction of access, while other functions like manipulation of data are not available.
- One can end up assigning privileges to the wrong users by improper use of GRANT and may not revoke unnecessary privileges using REVOKE, which can make the data vulnerable to unauthorized users.
- Handling permissions becomes challenging, especially since databases are large in the current world.
- DCL cannot perform INSERT, UPDATE, or DELETE statements.
- DCL is only useful when DML and DDL operations have already been performed.

5. Transaction Control Language(TCL)

TCL commands manage transactions in relational databases, ensuring data integrity and consistency. These commands are used to commit changes or roll back operations in case of errors.

Command	Description
COMMIT	Saves all changes made during the current transaction on a permanent basis. Some databases provide an auto-commit feature, which can be configured using settings.
ROLLBACK	Reverts changes made during the current transaction, ensuring no unwanted changes are saved.
SAVEPOINT	Sets a point within a transaction to which changes can be rolled back, allowing partial rollbacks

Benefits of SQL

- **Efficient:** Works fast even with large and complex data.
- **Standard:** Works on most database systems.
- **Scalable:** Can handle small to very large databases.
- **Flexible:** Supports advanced logic using extensions like PL/SQL or T-SQL.

Limitations of SQL

- **Complex:** Advanced tuning and optimization can be difficult.
- **Scalability Issues:** Not ideal for huge or unstructured data.
- **Different Versions:** Some SQL features vary by database.
- **Not Real-Time:** Traditional SQL is not built for real-time analytics.

Real-World Applications of SQL

- **E-Commerce:** Manage customer orders, product catalogs and inventory.
- **Healthcare:** Maintain patient records and appointment schedules.
- **Banking:** Analyze transaction histories and generate financial reports.
- **Web Development:** Power dynamic websites with user-specific content.
- **Machine Learning and Data Science:** Combine SQL databases with tools like Python, R and Tensor Flow to streamline machine learning workflows.

Table:

SQL Joins-

SQL Joins are used to combine rows from two or more tables based on a related column between them. They help in retrieving connected data stored across multiple tables.

Types of SQL Joins

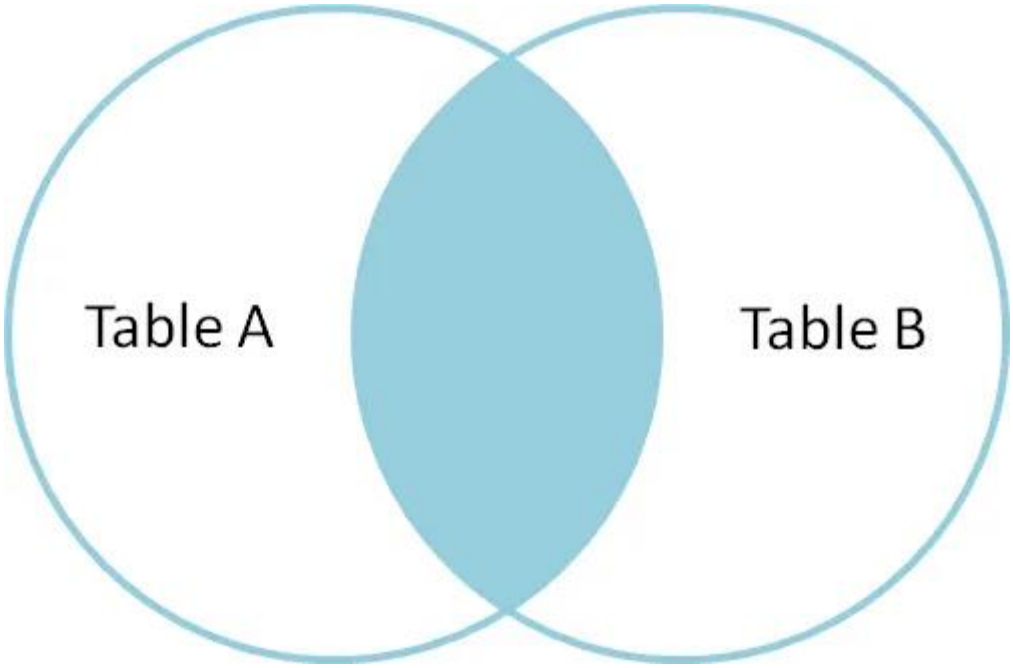
SQL joins are categorized into different types based on how rows from two tables are matched and combined.

1. SQL INNER JOIN

The INNER JOIN in SQL returns only the rows where there is a matching value in both tables. It combines records based on a related column and excludes non-matching rows.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,.... FROM table1  INNER JOIN
 table2 ON  table1.matching_column = table2.matching_column;
```



Inner Join

***Note**: We can also write JOIN instead of INNER JOIN. JOIN is same as INNER JOIN.*

Example of INNER JOIN:

Consider the two tables, Student and StudentCourse, which share a common column ROLL_NO. Using SQL JOINS, we can combine data from these tables based on their relationship, allowing us to retrieve meaningful information like student details along with their enrolled courses.

S_ID	NAME	ADDRESS
1	Liam	London
2	Sofia	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

Students Table

COURSE_ID	ROLL_NO
1	1
2	2
3	3
6	7

StudentCourse Table

Let's look at the example of INNER JOIN clause, and understand it's working. This query will show the names and age of students enrolled in different courses.

Query:

```
SELECT StudentCourse.COURSE_ID, Student.NAME, Student.AGE FROM Student
INNER JOIN StudentCourse
ON Student.ROLL_NO = StudentCourse.ROLL_NO;
```

Output:

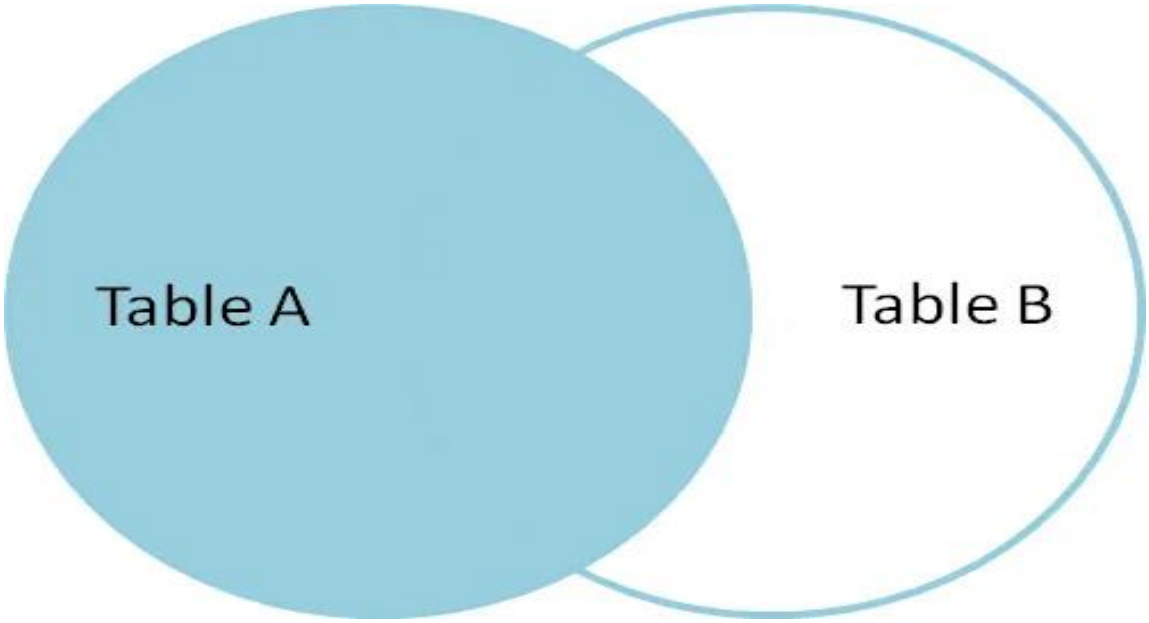
COURSE_ID	NAME	AGE
1	Liam Johnson	18
2	Emma Wilson	19
3	Noah Smith	20

2. SQL LEFT JOIN

A LEFT JOIN returns all rows from the left table, along with matching rows from the right table. If there is no match, NULL values are returned for columns from the right table. LEFT JOIN is also known as LEFT OUTER JOIN.

Syntax:

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
LEFT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Left JOIN

***Note**: We can also use LEFT OUTER JOIN instead of LEFT JOIN, both are the same.*

Example: In this example, the LEFT JOIN retrieves all rows from the Student table and the matching rows from the StudentCourse table based on the ROLL_NO column.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
LEFT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
Olivia Brown	
Ava Davis	

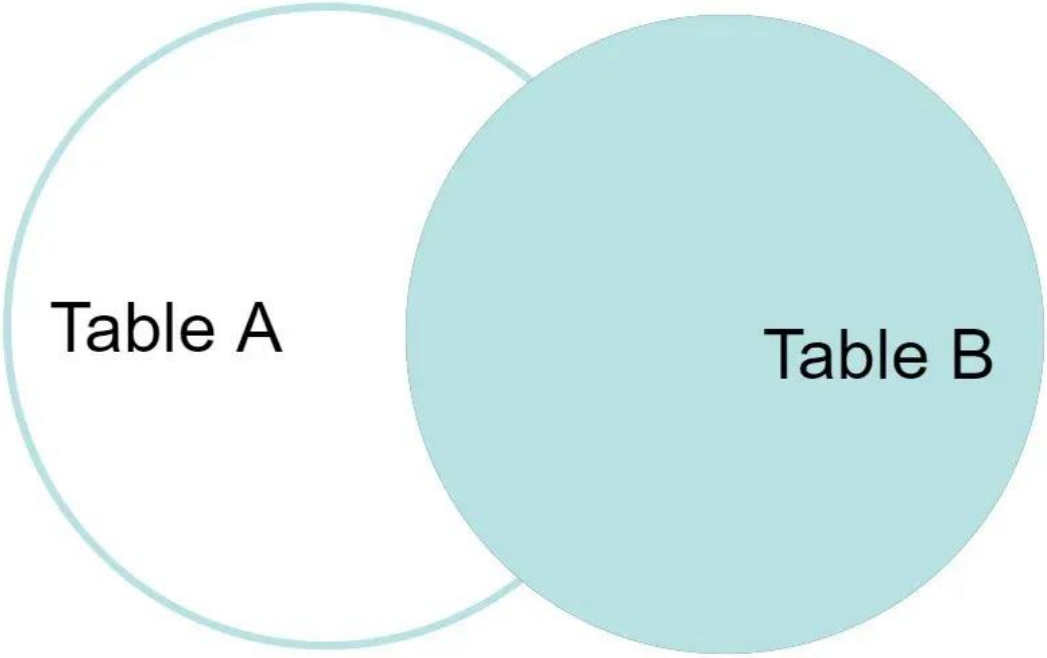
3. SQL RIGHT JOIN

RIGHT JOIN returns all the rows of the table on the right side of the join and matching rows for the table on the left side of the join. It is very similar to LEFT JOIN for the rows for which there is no matching row on the left side, the result-set will contain null. RIGHT JOIN is also known as RIGHT OUTER JOIN.

Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
```

```
RIGHT JOIN table2
ON table1.matching_column = table2.matching_column;
```



Right JOIN

***Note**: We can also use RIGHT OUTER JOIN instead of RIGHT JOIN, both are the same*

Example: In this example, the RIGHT JOIN retrieves all rows from the StudentCourse table and the matching rows from the Student table based on the ROLL_NO column.

Query:

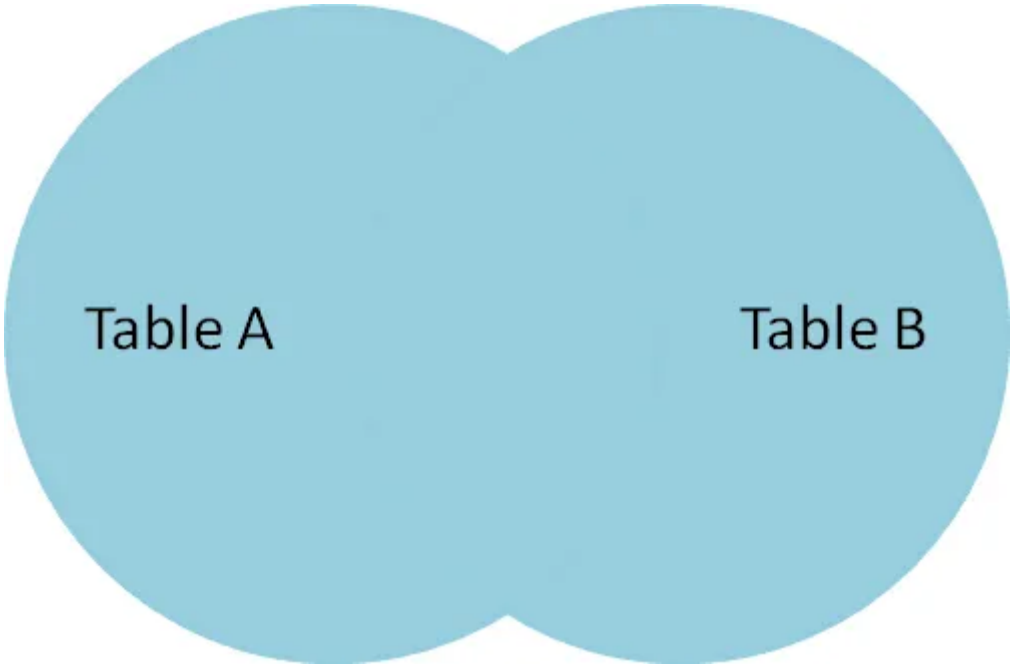
```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
RIGHT JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output:

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
	6

4. SQL FULL JOIN

FULL JOIN creates the result-set by combining results of both LEFT JOIN and RIGHT JOIN. The result-set will contain all the rows from both tables. For the rows for which there is no matching, the result-set will contain NULL values.



Syntax

```
SELECT table1.column1,table1.column2,table2.column1,....
FROM table1
FULL JOIN table2
ON table1.matching_column = table2.matching_column;
```

Example: This example uses a FULL JOIN (simulated in SQLite) to return all rows from both tables. Matching records appear together, while non-matching records still show up with NULL values for the missing fields.

Query:

```
SELECT Student.NAME,StudentCourse.COURSE_ID
FROM Student
FULL JOIN StudentCourse
ON StudentCourse.ROLL_NO = Student.ROLL_NO;
```

Output :

NAME	COURSE_ID
Liam Johnson	1
Emma Wilson	2
Noah Smith	3
Olivia Brown	
Ava Davis	
	6

5. SQL Natural Join

A Natural Join is a type of INNER JOIN that automatically joins two tables based on columns with the same name and data type. It returns only the rows where the values in the common columns match.

- It joins tables using common columns with the same name.

- It returns only rows where values in those columns match.
- The common column appears only once in the result.

Example: Look at the two tables below:

Employee Table:

Emp_id	Emp_name	Dept_id
1	Ram	10
2	Jon	30
3	Bob	50

Department Table:

Dept_id	Dept_name
10	IT
30	HR
40	TIS

Example: Find all Employees and their respective departments.

```
SELECT
   Emp_name,
   Dept_name
FROM Employee
NATURAL JOIN Department;
```

Output:

Emp_name	Dept_name
Ram	IT
Jon	HR

Views -

Views in MySQL are indeed **"virtual tables"** that are used to view data from one or more tables. Views do not have their data but rather store data virtually, consisting of rows and columns. Views are very helpful in restricting access to your application's critical data to third-party users. Views in MySQL can be created by selecting some/all columns and some/all rows of a table by filtering out the rows based on some condition(s).

What is the Benefit of Using Views in MySQL?

Views help particularly in the following ways:

1. **Simplicity:** Instead of writing complex joins & queries, views provide a way of writing simple SELECT statements.

2. **Enhanced Security:** Views expose only the data to the third-party apps and hide the internal details like table structure, attributes, etc, thus adding extra security.
3. **Consistency:** By writing views instead of common queries, we can write a view that avoids multiple declarations & definitions of the same queries and eventually provides a centralized way.

Just like the normal tables, you can perform operations like create, update, drop, etc., on the views. We will look at their syntax along with examples now.

Types of Views –

1. **Simple View**
   - Created from single table
   - No GROUP BY, DISTINCT, JOIN

2. **Complex View**
   - Created from multiple tables
   - Can use JOIN, GROUP BY, aggregate functions

What is MySQL Command Line Client?

MySQL Command Line Client is a simple and elegant **SQL shell** with inline editing ability. It is basically a Non-GUI based approach to query and interact with our MySQL database. So in simpler terms, it is a tool that permits sending MySQL queries to MySQL database from the command line i.e. shell. This is typically useful when we cannot install GUI-based tools like MySQL Workbench for interaction with MySQL database like when we are having MySQL database on a remote server.
To connect to the local MySQL database using the command line, follow the below steps:

1. Open the command prompt/shell.
2. Run the command **mysql -u root -p**
3. Enter the password, then you will be connected to your MySQL database.

➢ Let us consider the following schemas for the examples in the following article.

StudentDetails Table:

sid	sname	age	university
1	Girish	24	IIT Hyderabad
2	Aaditya	24	SRM University
3	Aashish	23	IIT Hyderabad
4	John	25	Mumbai University

sid	sname	age	university
5	Shruti	24	IIT Hyderabad
6	Leena	25	Mumbai University

CourseDetails Table:

cid	cname	ratings	price
1	Python Fundamentals	4.6	2999
2	Machine Learning	4.3	1999
3	DSA A-Z	4.9	5999
4	Competitive Programming	4.7	4999

EnrolledIn Table:

sid	cid
1	3
1	4
2	1
2	3
3	3
4	1

Create View in MySQL

A view in MySQL can be created based on a single table or multiple tables.
The **CREATE VIEW** statement is used to create a view in MYSQL.

1. Create a View Based On Single Table

Syntax:

CREATE VIEW <view_name> AS
SELECT <column1>, <column2>....., <columnN>
FROM <table-name>
WHERE [conditions];

Example:

Let's us create a view named "**IITHyderabadStudentsView**" from the StudentDetails table. This view selects the students from the StudentDetails table who study in "IIT Hyderabad" university and outputs their details like student id, name, and age.

```
CREATE VIEW IITHyderabadStudentsView AS
SELECT sid, sname, age
FROM StudentDetails
WHERE university = "IIT Hyderabad";
```

Output:

```
mysql> CREATE VIEW IITHyderabadStudentsView AS
    -> SELECT sid, sname, age
    -> FROM StudentDetails
    -> WHERE university = "IIT Hyderabad";
Query OK, 0 rows affected (0.00 sec)
```

Create VIEW - Single table

Now to view the tuples in the IITHyderabadStudentsView, we will query the view the just like how we query a normal table.

```
SELECT * FROM IITHyderabadStudentsView;
```

Output:

```
mysql> SELECT * FROM IITHyderabadStudentsView;
+-----+---------+-----+
| sid | sname   | age |
+-----+---------+-----+
|   1 | Girish  |  24 |
|   3 | Aashish |  23 |
|   5 | Shruti  |  24 |
+-----+---------+-----+
3 rows in set (0.07 sec)
```

Query Output - Create VIEW - Single table

2. Create a View Based On Multiple Tables With JOIN Clause

We can create a view by joining multiple tables using the JOIN clause and additionally using WHERE keyword or GROUP BY, HAVING clauses.

Syntax:

CREATE VIEW <view_name> AS
SELECT <column1>, <column2>....., <columnN>
FROM <table1>
[INNER | LEFT | RIGHT | FULL] JOIN <table2> ON <joining-column>
WHERE [condition1 | condition2 | .....];

Example:

Let's us create a view named "**PythonEnrolledView**" using the StudentDetails, CourseDetails, and EnrolledIn table. This view outputs the students who are enrolled in "Python Fundamentals" course the details as student id, name, and age.

```
CREATE VIEW PythonEnrolledView AS
SELECT S.sid, S.sname, S.age
FROM StudentDetails S JOIN EnrolledIn E ON S.sid = E.sid
JOIN CourseDetails C ON C.cid = E.cid
WHERE C.cname = "Python Fundamentals";
```

Output:

```
mysql> CREATE VIEW PythonEnrolledView AS
    -> SELECT S.sid, S.sname, S.age
    -> FROM StudentDetails S JOIN EnrolledIn E ON S.sid = E.sid
    -> JOIN CourseDetails C ON C.cid = E.cid
    -> WHERE C.cname = "Python Fundamentals";
Query OK, 0 rows affected (0.01 sec)
```

Create VIEW - Multiple table using JOIN

Now to view the tuples in the PythonEnrolledView, we will again query the view the just like how we query a normal table.

```
SELECT * FROM PythonEnrolledView;
```

Output:

```
mysql> SELECT * FROM PythonEnrolledView;
+-----+----------+-----+
| sid | sname    | age |
+-----+----------+-----+
|   2 | Aaditya  |  24 |
|   4 | John     |  25 |
+-----+----------+-----+
2 rows in set (0.01 sec)
```

Query Output - Create VIEW - Multiple tables (JOIN)

Update View in MySQL

There are certain conditions that need to be satisfied to update a view. If any one of these conditions is not met, then we are not allowed to update the view.

1. The View should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.
2. The View should not be created using nested queries or complex queries.
3. The View should have all **NOT NULL** values.
4. The **SELECT** statement should not have the **DISTINCT** keyword.
5. The SELECT statement which is used to create the view should not include **GROUP BY** clause or **ORDER BY** clause.

1. Update View Definition/Structure

To update the view for adding or remove columns and rows by changing WHERE clause condition, we can use **CREATE OR REPLACE VIEW** statement.

Syntax:

CREATE OR REPLACE VIEW <view_name> AS
SELECT <column1>, <column2>, ..., <columnN>
FROM <table_name>
WHERE [condition];

Example:

Suppose we want to update the view IITHyderabadStudentsView we created above and delete the column sid from this view from StudentDetails table, we can do this as follows:

```
CREATE OR REPLACE VIEW IITHyderabadStudentsView AS
SELECT sname, age
FROM StudentDetails
WHERE university = "IIT Hyderabad";
```

Output:

```
mysql> CREATE OR REPLACE VIEW IITHyderabadStudentsView AS
    -> SELECT sname, age
    -> FROM StudentDetails
    -> WHERE university = "IIT Hyderabad";
Query OK, 0 rows affected (0.02 sec)
```

UPDATE VIEW Definition

2. Insert Into View

To insert the new row into the view, we can do it in a similar way just like how we do it for normal tables.

Syntax:

```
INSERT INTO <view_name>(<column1>, <column2>,
<column3>,.........)
VALUES(<value1>, <value2>, <value3>,...........);
```

Example:

Let us insert a new row in the view IITHyderabadStudentsView which we have created above in the example of "Create View-based On Single Table".

```
INSERT INTO IITHyderabadStudentsView(sid, sname, age)
VALUES(7, "Tenali Rama", 26);

SELECT * FROM IITHyderabadStudentsView;
SELECT * FROM StudentDetails;
```

Output:

You can see from the image below that inserting a row into view inserted the row into our original table as well. To see the changes, you can query the data from original table before and after insertion into the view.

```
mysql> SELECT * FROM StudentDetails;
+-----+---------+-----+-------------------+
| sid | sname   | age | university        |
+-----+---------+-----+-------------------+
|   1 | Girish  |  24 | IIT Hyderabad     |
|   2 | Aaditya |  24 | SRM University    |
|   3 | Aashish |  23 | IIT Hyderabad     |
|   4 | John    |  25 | Mumbai University |
|   5 | Shruti  |  24 | IIT Hyderabad     |
|   6 | Leena   |  25 | Mumbai University |
+-----+---------+-----+-------------------+
6 rows in set (0.00 sec)

mysql> INSERT INTO IITHyderabadStudentsView(sid, sname, age) VALUES(7, "Tenali Rama", 26);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM StudentDetails;
| sid | sname       | age | university        |
+-----+-------------+-----+-------------------+
|   1 | Girish      |  24 | IIT Hyderabad     |
|   2 | Aaditya     |  24 | SRM University    |
|   3 | Aashish     |  23 | IIT Hyderabad     |
|   4 | John        |  25 | Mumbai University |
|   5 | Shruti      |  24 | IIT Hyderabad     |
|   6 | Leena       |  25 | Mumbai University |
|   7 | Tenali Rama |  26 | IIT Hyderabad     |
+-----+-------------+-----+-------------------+
7 rows in set (0.00 sec)
```

Insert Into View

Output:

```
mysql> SELECT * FROM IITHyderabadStudentsView;
+-----+-------------+-----+
| sid | sname       | age |
+-----+-------------+-----+
|   1 | Girish      |  24 |
|   3 | Aashish     |  23 |
|   5 | Shruti      |  24 |
|   7 | Tenali Rama |  26 |
+-----+-------------+-----+
4 rows in set (0.01 sec)
```

SELECT * FROM View

3. Delete From View

To insert the new row into the view, we can do it in a similar way just like how we do it for normal tables. The syntax is:

```
DELETE FROM <view_name> WHERE [condition];
```

Example:

Let us delete a row in the view IITHyderabadStudentsView which we have created above in the example of "Create View-based On Single Table". Let us delete the details of student whose name is "Tenali Rama".

```
DELETE FROM IITHyderabadStudentsView WHERE sname = "Tenali
Rama";
```

Output:

The data in the view before deletion is performed.

```
mysql> SELECT * FROM IITHyderabadStudentsView;
+-----+-------------+-----+
| sid | sname       | age |
+-----+-------------+-----+
|   1 | Girish      |  24 |
|   3 | Aashish     |  23 |
|   5 | Shruti      |  24 |
|   7 | Tenali Rama |  26 |
+-----+-------------+-----+
4 rows in set (0.01 sec)
```

SELECT * FROM View

The data in the view after deletion is performed.

```
mysql> DELETE FROM IITHyderabadStudentsView WHERE sname = "Tenali Rama";
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM IITHyderabadStudentsView;
+-----+---------+-----+
| sid | sname   | age |
+-----+---------+-----+
|   1 | Girish  |  24 |
|   3 | Aashish |  23 |
|   5 | Shruti  |  24 |
+-----+---------+-----+
3 rows in set (0.00 sec)
```

DELETE FROM VIEW

Drop View in MySQL

Suppose now there is no need of the created view anymore? So, we want to delete it now. MySQL allows to deletion an already existing view. We can drop a view using the DROP statement.

Syntax:

DROP VIEW <view_name>;

Example:

Let's suppose we want to delete the view named "PythonEnrolledView" that we created above, which can be done as follows:

```
DROP VIEW PythonEnrolledView;
```

Output:

```
mysql> DROP VIEW PythonEnrolledView;
Query OK, 0 rows affected (0.02 sec)
```

DROP View

Aggregation functions –

SQL Aggregate Functions allow summarizing large sets of data into meaningful results, making it easier to analyze patterns and trends across many records. They return a single output value after processing multiple rows in a table.

- Perform calculations like totals, averages, minimum or maximum values on data.
- Ignore NULL values in most functions except COUNT(*), improving result accuracy.
- Work with clauses such as GROUP BY, HAVING and ORDER BY for analysis.

Example: First, we create a demo SQL database and table, on which we use the Aggregate functions.

EmpID	Name	Salary
1	John	5000
2	Sara	
3	Michael	8000
4	Emma	5000

Employee Table

Query:

```
SELECT SUM(Salary) FROM Employee;
```

Output:

SUM(Salary)
18000

Syntax:

```
AGGREGATE_FUNCTION(column_name)
```

Aggregate Functions in SQL

Below are the most frequently used aggregate functions in SQL.

1. Count()

It is used to count the number of rows in a table. It helps summarize data by giving the total number of entries. It can be used in different ways depending on what you want to count:

- **COUNT(*):** Counts all rows.
- **COUNT(column_name):** Counts non-NULL values in the specified column.
- **COUNT(DISTINCT column_name):** Counts unique non-NULL values in the column.

Query:

```
-- Total number of records in the table
SELECT COUNT(*) AS TotalRecords FROM Employee;

-- Count of non-NULL salaries
SELECT COUNT(Salary) AS NonNullSalaries FROM Employee;

-- Count of unique non-NULL salaries
SELECT COUNT(DISTINCT Salary) AS UniqueSalaries FROM Employee;
```

Output:

TotalRecords
4

NonNullSalaries
3

UniqueSalaries
2

- COUNT(*) returns the total number of rows in the table, including rows with NULL values.
- COUNT(Salary) counts only the rows where Salary is not NULL.
- COUNT(DISTINCT Salary) counts unique non-NULL salary values, ignoring duplicates.

2. SUM()

It is used to calculate the total of a numeric column. It adds up all non-NULL values in that column for Example, SUM(column_name) returns sum of all non-NULL values in the specified column.

Query:

```
-- Calculate the total salary
SELECT SUM(Salary) AS TotalSalary FROM Employee;

-- Calculate the sum of unique salaries
SELECT SUM(DISTINCT Salary) AS DistinctSalarySum FROM
Employee;
```

Output:

TotalSalary
18000

DistinctSalarySum
13000

- SUM(Salary) adds all non-NULL salary values to get the total salary amount.
- SUM(DISTINCT Salary) adds only unique non-NULL salary values, avoiding duplicates.
- NULL values are ignored in both SUM calculations.

3. AVG()

- It is used to calculate average value of a numeric column. It divides sum of all non-NULL values by the number of non-NULL rows for Example,

AVG(column_name) returns average of all non-NULL values in the specified column.

Query:

```
-- Calculate the average salary
SELECT AVG(Salary) AS AverageSalary FROM Employee;

-- Average of distinct salaries
SELECT AVG(DISTINCT Salary) AS DistinctAvgSalary FROM
Employee;
```

Output:

AverageSalary
6000

DistinctAvgSalary
6500

- AVG(Salary) calculates the average of all non-NULL salary values.
- AVG(DISTINCT Salary) computes the average only from unique non-NULL salary values.
- Both ignore NULL values when performing the calculation.

4. MIN() and MAX()

The MIN() and MAX() functions return the smallest and largest values, respectively, from a column.

Query:

```
-- Find the highest salary
SELECT MAX(Salary) AS HighestSalary FROM Employee;

-- Find the lowest salary
SELECT MIN(Salary) AS LowestSalary FROM Employee;
```

Output:

HighestSalary
8000

LowestSalary
5000

- MAX(Salary) returns the highest non-NULL salary value from the Employee table.
- MIN(Salary) returns the lowest non-NULL salary value from the Employee table.
- Both functions ignore NULL values while determining the result.

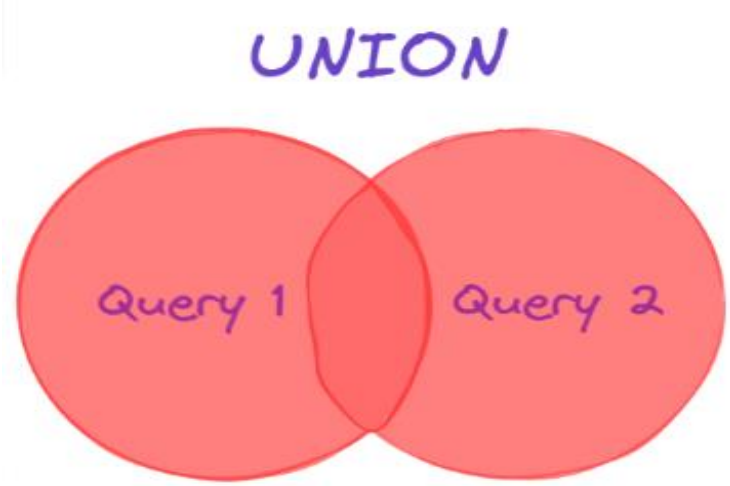
Set operations-

- Set operations in DBMS allow you to combine the results of two or more SQL queries. These operations are based on set theory in mathematics and work with rows of data.
- Set operations form the foundation of SQL and enable us to combine, compare, and filter data from multiple sources. These operations are indispensable for tasks ranging from data integration and cleansing to advanced analytics and reporting.
- Set operations in SQL are techniques for combining or comparing the results of two or more SELECT statements. They act like mathematical set operations, letting us find the union, intersection, or difference between the rows returned by our queries. This makes them indispensable when analyzing data from multiple sources or perspectives.
- The most common set operations in SQL are:

  A. UNION
  B. UNION ALL
  C. INTERSECT
  D. EXCEPT (or MINUS)

A. UNION

- The UNION operator combines the results of two or more SELECT queries into a single result set, removing duplicate rows by default. It Combines the results of two queries and removes duplicate rows.



Syntax

```
SELECT column1, column2 FROM table1
UNION
SELECT column1, column2 FROM table2;
```

Example:

```
SELECT name FROM students_2023
UNION
SELECT name FROM students_2024;
```

comp1_employees Table :

employee_id	employee_name	employee_city
1	Ajay	Surat
2	Vijay	Goa
3	Sanjay	Ahmedabad

comp2_employees:

employee_id	employee_name	employee_city
1	Vishnu	Raipur
2	Vijay	Goa
3	Mohak	Gurugram

```
SELECT * FROM comp1_employees
UNION
SELECT * FROM comp2_employees;
```

The result of the above query is shown below:

employee_id	employee_name	employee_city
1	Ajay	Surat
1	Vishnu	Raipur
2	Vijay	Goa
3	Mohak	Gurugram
3	Sanjay	Ahmedabad

- As shown in the above code snippet, there are two SELECT queries, and a UNION operator is used. The first SELECT query will fetch the records from comp1_employees, and the second SELECT query will fetch the records from comp2_employees, and the UNION operation is performed with the results of both the query.
- After performing the UNION operation with both the tables, all the records from the comp1_employees table and comp2_employees table are displayed except for the duplicate data, i.e., employee_id -- 2 and 2 are duplicates. Hence, they are displayed only one time.

Key Points:

- Remove duplicates automatically.
- Both queries must have the same number of columns and compatible data types.

B. UNION ALL

- Combines the results of two queries and does not remove duplicates.
- UNION and UNION ALL are similar in their functioning, but there is a slight difference. UNION ALL is also used to combine the results of two or more SELECT statements. To successfully execute the operation of Union All, the number of columns and the data type must be the same in both tables. After performing the UNION ALL operation, the duplicate rows will not be eliminated from the results, and all the data is displayed in the result without removing the duplication.

Syntax:

```
SELECT column1, column2 FROM table1
UNION ALL
SELECT column1, column2 FROM table2;
```

Example:

Now, let's take an example to clearly understand how the UNION ALL operator works.

```
SELECT * FROM comp1_employees
UNION ALL
SELECT * FROM comp2_employees;
```

employee_id	employee_name	employee_city
1	Ajay	Surat
2	Vijay	Goa
3	Sanjay	Ahmedabad
1	Vishnu	Raipur
2	Vijay	Goa
3	Mohak	Gurugram

This query retrieves all student names, including duplicates.

- As shown in the above code snippet, there are two SELECT queries, and a UNION All operator is used. The first SELECT query will fetch the records from comp1_employees, and the second SELECT query will fetch the records from comp2_employees, and the UNION ALL operation is performed with the results of both the query.
- After performing the UNION ALL operation with both tables, all the records from the comp1_employees table and comp2_employees table are displayed. Since it's a UNION ALL operation, all the records are displayed, including the duplicate words, which is not the case in the UNION operation.

Key Points:

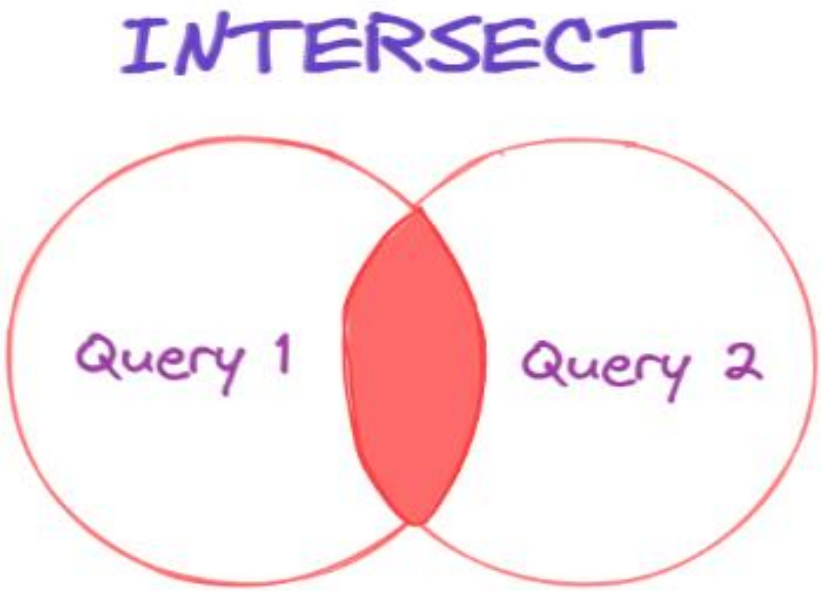
- Faster than UNION because it doesn't check for duplicates.
- Used when duplicates are acceptable.

C. INTERSECT

- The INTERSECT operator allows you to find the results that exist in both queries. To successfully execute the operation of INTERSECT, the number of columns and the data type must be the same in both tables. After performing the INTERSECT

operation, the data/records which are common in both the SELECT statements are returned.

- Retrieves rows that are common to both queries.



Syntax:

```
SELECT column1, column2 FROM table1 I
NTERSECT
SELECT column1, column2 FROM table2;
```

Example:

Now, let's take an example to clearly understand how the INTERSECT operator works.

```
SELECT * FROM comp1_employees
INTERSECT
SELECT * FROM comp2_employees;
```

The result of the above query is shown below:

employee_id	employee_name	employee_city
2	Vijay	Goa

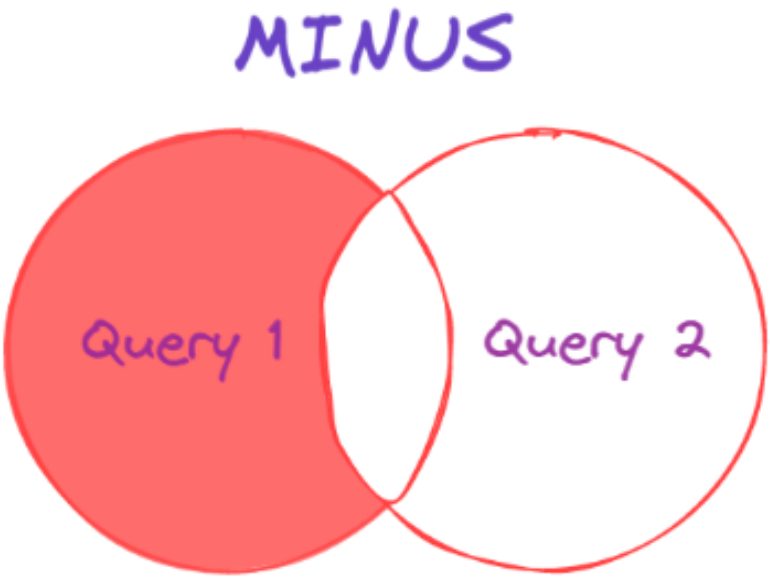
- This query retrieves student names that appear in both tables.
- As shown in the above code snippet, there are two SELECT queries, and an INTERSECT operator is used. The first SELECT query will fetch the records from comp1_employees, and the second SELECT query will fetch the records from comp2_employees, and the INTERSECT operation is performed with the results of both the query.
- After performing the INTERSECT operation with both tables, all the data/records that are common from the comp1_employees table and comp2_employees table are displayed. As you can see in the result above, the INTERSECT gets the results that exist in both queries. And the employee_id -- 2 and 77 exist in both tables; hence they are displayed in the final result.

Key Points:

- Shows only matching rows.
- Removes duplicates from the final result.

D. EXCEPT (or MINUS in some databases)

- Retrieves rows from the first query that are not present in the second query.
- The MINUS operator allows you to filter out the results which are present in the first query but absent in the second query. To successfully execute the MINUS operation, the number of columns and the data type must be the same in both tables. After performing the MINUS operation, the data/records which are not present in the second SELECT statement or query are displayed.
  Note: The MINUS operator is supported only in Oracle databases. For other databases like SQLite, PostgreSQL, SQL server, you can use EXCEPT operator to perform similar operations.



Syntax:
SELECT column1, column2 FROM table1 EXCEPT
SELECT column1, column2 FROM table2;

Example:

Example:

```
SELECT * FROM comp1_employees
EXCEPT
SELECT * FROM comp2_employees;
```

The result of the above query is shown below:

employee_id	employee_name	employee_city
1	Ajay	Surat
3	Sanjay	Ahmedabad

- This query retrieves student names that are in students_2023 but not in students_2024.

Key Points:

- The order of queries matters.
- Removes duplicates in the result.

Important Rules for Set Operations

- Both queries must have the same number of columns.
- Columns must have compatible data types.
- The column order in both queries must match.
- These operations are widely used for comparing datasets and merging results.

Operation	Purpose	Duplicates Allowed?	Example Use Case
UNION	Combines results and removes duplicates.	No	Combine unique student names from two batches.
UNION ALL	Combines results and keeps duplicates.	Yes	Combine student lists, showing duplicate entries.
INTERSECT	Retrieves only common rows between two queries.	No	Find students present in both batches.
EXCEPT	Retrieves rows from the first query that are not in the second query.	No	Find students only in one batch, not the other.

SQL Operators-

SQL operators are symbols or keywords used to perform operations on data in SQL queries.

- Perform operations like calculations, comparisons, and logical checks.
- Enable filtering, calculating, and updating data in databases.
- Essential for query optimization and accurate data management.

1. Arithmetic Operators

- Arithmetic operators in SQL are used to perform mathematical operations on table data. These operators help in calculating totals, differences, percentages, and other numeric transformations directly in queries.

  These operations can be applied to:

  - A single column
  - Two or more columns
  - Constant values with column data

List of Arithmetic Operators in SQL

Operator	Description
+	Addition
-	Subtraction
/	Division
*	Multiplication
%	Modulus (Remainder)

1. Addition (+)

The addition operator is used to sum values. Performs addition between:

- Column and constant
- Two columns

Example 1: Addition with constant

```
SELECT employee_id, employee_name, salary,
       salary + 100 AS "salary + 100"
FROM addition;
```

Output:

employee_id	employee_name	salary	salary+100
1	Alex	25000	25100
2	RR	55000	55100
3	JPM	52000	52100
4	GGSHMR	12312	12412

2. Subtraction (-)

The subtraction operator deducts one value from another. Performs subtraction between:

- Column and constant
- Two columns

Example 1: Subtracting a constant

```
SELECT employee_id, employee_name, salary,
       salary - 100 AS "salary - 100"
FROM subtraction;
```

Output:

employee_id	employee_name	salary	salary-100
12	Finch	15000	14900
22	Peter	25000	24900
32	Warner	5600	5500
42	Watson	90000	89900

3. Multiplication (*)

The multiplication operator multiplies values by constants or other column values. It multiplies:

- Column and constant
- Two columns

Example 1: Multiplying with a constant

```
SELECT employee_id, employee_name, salary,
       salary * 100 AS "salary * 100"
FROM addition;
```

Output:

employee_id	employee_name	salary	salary*100
1	Finch	25000	2500000
2	Peter	55000	5500000

employee_id	employee_name	salary	salary*100
3	Warner	52000	5200000
4	Watson	12312	1231200

4 . Division (/)

The division operator divides one value by another. Example for division is similar to multiplication but returns **quotient** instead of product.

Example:

```
SELECT employee_id, employee_name, salary,
       salary / 100 AS "salary / 100"
FROM addition;
```

Output:

employee_id	employee_name	salary	salary/100
1	Finch	25000	250
2	Peter	55000	550
3	Warner	52000	520
4	Watson	12312	123.12

6. Modulus (%)

The modulus operator returns the remainder of a division.

Useful for:

- Even/Odd check
- Pattern-based calculations

Example 1: Modulus with constant

```
SELECT employee_id, employee_name, salary,
       salary % 25000 AS "salary % 25000"
FROM addition;
```

Output:

employee_id	employee_name	salary	salary%25000

employee_id	employee_name	salary	salary%25000
1	Finch	25000	0
2	Peter	55000	5000
3	Warner	52000	2000
4	Watson	12312	12312

QL Comparison Operators are used to compare two values and filter the result set based on a specific condition.

- Used in the WHERE clause to filter records based on conditions
- Help check for equality, range, or inequality
- Make SQL queries powerful and precise
- Examples: =, >, <, >=, <=, <>

2. Comparison Operators

The below table shows all comparison operators in SQL :

Operator	Description
=	The SQL Equal Operator checks if the values of two operands are equal.
!=	The SQL Not Equal Operator checks if the values of two operands are not equal.
>=	The SQL Greater Than Equals to Operator checks if the value of the left operand is greater than or equal to the value of the right operand.
<	The SQL Less Than Operator checks if the value of the left operand is less than the value of the right operand.
>	The SQL Greater Than Operator checks if the value of the left operand is greater than the value of the right operand.
<=	The SQL Less Than Equals to Operator checks if the value of the left operand is less than or equal to the value of the right operand.

Syntax

```
SELECT * FROM TABLE_NAME WHERE
ATTRIBUTE CONDITION_OPERATOR GIVEN_VALUE;
```

3. Logical Operators

SQL Logical Operators are used to test conditions in queries, returning results as TRUE, FALSE, or UNKNOWN. They help in combining, negating, and comparing conditions, enabling precise data retrieval and filtering.

- Combine multiple conditions in a query.
- Control query execution flow.
- Support complex filtering and comparison.
- Return boolean results for decision-making.

Example: First, we will create a demo SQL database and table, on which we will use the Logical Operators command.

emp_id	emp_name	emp_city	emp_country
101	Liam Johnson	New York	USA
102	Emma Williams	London	UK
103	Noah Smith	Toronto	Canada
104	Olivia Brown	Sydney	Australia
105	Lucas Garcia	Madrid	Spain
106	Isabella Rossi	Rome	Italy

Below is the comprehensive list of SQL Logical Operators along with their meanings, detailed explanations, and practical examples:

1. AND Operator

The AND operator is used to combine two or more conditions in an SQL query. It returns records only when all conditions specified in the query are true. This operator is commonly used when filtering data that must satisfy multiple criteria simultaneously.

Example

Retrieve the records of employees from the employees table who are located in 'London' and belong to 'UK', ensuring that both conditions are met.

Query:

```
SELECT *

FROM employee

WHERE emp_city = 'London' AND emp_country = 'UK';
```

Output

emp_id	emp_name	emp_city	emp_country
102	Emma Williams	London	UK

Explanation:

In the output, both conditions (emp_city = 'London' and emp_country = 'UK') are satisfied for the listed employees, so these records are returned by the query.

2. OR Operator

The OR operator combines multiple conditions in a SQL query and returns TRUE if at least one of the conditions is satisfied. It is ideal for situations where you want to retrieve records that meet any of several possible conditions.

Example

Retrieve the records of employees from the employee table who are either from 'USA' or have 'UK' as their country.

Query

```
SELECT *

FROM employee

WHERE emp_city = 'London' OR emp_country = 'USA';
```

Output

emp_id	emp_name	emp_city	emp_country
101	Liam Johnson	New York	USA
102	Emma Williams	London	UK

Explanation:

In this case, the output includes employees from 'USA' as well as those who have 'UK' as their country, even if they are from different cities. The query returns all records where at least one of the conditions is true.

3. NOT Operator

The NOT operator is used to reverse the result of a condition, returning TRUE when the condition is FALSE. It is typically used to exclude records that match a specific condition, making it useful for filtering out unwanted data.

Example

Retrieve the records of employees from the employee table whose city names do not start with the letter 'S'.

Query:

```
SELECT *

FROM employee

WHERE emp_city NOT LIKE 'S%';
```

Output

emp_id	emp_name	emp_city	emp_country
101	Liam Johnson	New York	USA
102	Emma Williams	London	UK
103	Noah Smith	Toronto	Canada
105	Lucas Garcia	Madrid	Spain
106	Isabella Rossi	Rome	Italy

Explanation:

In this query, the NOT operator negates the LIKE condition. The LIKE operator is used to match patterns in string data, and the 'S%' pattern matches any city name that starts with the letter 'S'. By using the NOT operator, we exclude cities starting with 'S' from the result set.

4. Special Operators- Used for advanced filtering.

1. IN Operator

The IN operator streamlines checking whether a value matches any value from a specified list, making queries more concise and readable than using multiple OR conditions. It is especially useful for filtering results by several possible values in a column, helping reduce query complexity

Example

Retrieve the records of employees from the employee table who are located in either 'London' and 'Rome'

Query:

```
SELECT *

FROM employee

WHERE emp_city IN ('London', 'Rome');
```

Output

emp_id	emp_name	emp_city	emp_country
102	Emma Williams	London	UK
106	Isabella Rossi	Rome	Italy

Explanation:

In this query, the IN operator checks if the value of the emp_city column matches any value in the list ('London', 'Rome'). The query returns all employees who are located in either of these two cities.

2. BETWEEN Operator

The BETWEEN operator in SQL allows us to test if a value or expression lies within a specified range.

- **Inclusive Range** – Includes both the lower and upper limits in the result set.

- **Versatile Use** – Works with numbers, dates, and text values.
- **Efficient Filtering** – Ideal for selecting data within a defined range.

Example

Retrieve the records of employees from the employee table whose emp_id values fall within the range of 102 to 105 (inclusive).

Query:

```
SELECT *

FROM employee

WHERE emp_id BETWEEN 102 AND 105;
```

Output

emp_id	emp_name	emp_city	emp_country
102	Emma Williams	London	UK
103	Noah Smith	Toronto	Canada
104	Olivia Brown	Sydney	Australia
105	Lucas Garcia	Madrid	Spain

Explanation:

In this query, the BETWEEN operator is used to filter employees with emp_id values ranging from 102 to 105. Since the BETWEEN operator is inclusive, employees with emp_id values of 102, 103, 104, and 105 will be included in the result set.

3. LIKE Operator

The LIKE operator in SQL is used in the WHERE clause to search for a specified pattern in a column. It is particularly useful when we want to perform pattern matching on string data. The LIKE operator works with two main wildcards:

- **%**: Represents zero or more characters. It allows matching any sequence of characters in the string.
- _: Represents exactly one character. It is used when you want to match a specific number of characters at a given position.

Example

Retrieve the records of employees from the employee table whose city names start with the letter 'M'.

Query:

```
SELECT *

FROM employee

WHERE emp_city LIKE 'M%';
```

Output

emp_id	emp_name	emp_city	emp_country
105	Lucas Garcia	Madrid	Spain

Explanation:
In this case, the output includes only those employees whose emp_city starts with 'M'. The % wildcard ensures that the query matches any city name starting with the specified letter, regardless of how many additional characters follow it.

4. ALL Operator

The ALL operator in SQL is used to compare a value to all values returned by a subquery.

- Returns TRUE only if the condition is TRUE for all values returned by the subquery.
- Commonly used with SELECT, WHERE, and HAVING clauses.
- Ensures a value meets the specified condition when compared to an entire result set.

Example

Retrieve the records of employees whose emp_id is equal to all emp_id values in the employees table where the emp_city is 'London'.

Query:

```
SELECT *
FROM employee
WHERE emp_city = 'London'
  AND emp_id >= ALL (
      SELECT emp_id
      FROM employee
      WHERE emp_city = 'London'
  );
```

Output

emp_id	emp_name	emp_city	emp_country
102	Emma Williams	London	UK

Explanation:
The query checks whether emp_id in the outer query is equal to every emp_id from the subquery (which retrieves emp_id values from employees in 'London'). In this case, the output will include employees whose emp_id matches all the values in the subquery.

5. ANY Operator

The ANY operator in SQL is used to compare a value with the results of a subquery.

- Returns TRUE if the value meets the condition with any value from the subquery results.
- Useful for checking if a value matches at least one item in a set.
- Provides flexibility when comparing against multiple possible values.

Example

Retrieve the records of employees whose emp_id matches any of the emp_id values in the employees table where the emp_city is 'Rome'.

Query:

```
SELECT *
FROM employee
WHERE emp_id = ANY (
    SELECT emp_id
    FROM employee
    WHERE emp_city = 'Rome'
);
```

Output

emp_id	emp_name	emp_city	emp_country
106	Isabella Rossi	Rome	Italy

Explanation:

The output shows employees whose emp_id matches at least one emp_id from employees in 'Rome' (106).

6. EXISTS Operator

The EXISTS operator in SQL is used to check whether a subquery returns any rows.

- Returns TRUE if the subquery returns one or more rows.
- Commonly used with SELECT, UPDATE, INSERT, and DELETE statements.
- Helps check if any matching rows exist for a given condition.
- Often used in correlated subqueries referencing the outer query.

Example

Retrieve the names of employees from the employee table if there are any employees in the employee table who are located in 'London'.

Query

```
SELECT emp_name
FROM employee
WHERE EXISTS (
    SELECT emp_id
    FROM employee
    WHERE emp_city = 'London'
);
```

Output

emp_name
Liam Johnson
Emma Williams
Noah Smith
Olivia Brown
Lucas Garcia
Isabella Rossi

Explanation:

The EXISTS operator returns TRUE if the subquery finds any employees from Patna, so all employee names are included. The query lists all employees as long as at least one is from London.

7. SOME Operator

- Used with comparison operators (<, >, =, <=, etc.) to compare a value against subquery results.
- Returns TRUE if the condition is satisfied for at least one value from the subquery.
- Useful for matching a value against multiple possible results instead of just one.

Example

Retrieve the records of employees from the employee table where the emp_id is less than any of the emp_id values from employees located in 'London'.

Query:

```
SELECT *

FROM employee

WHERE emp_id < SOME (

    SELECT emp_id

    FROM employee

    WHERE emp_city = 'Madrid'

);
```

Output

emp_id	emp_name	emp_city	emp_country
101	Liam Johnson	New York	USA
102	Emma Williams	London	UK
103	Noah Smith	Toronto	Canada
104	Olivia Brown	Sydney	Australia

Explanation:
The query returns employees whose emp_id is less than at least one emp_id of employees from London.

5. Set Operators

Used to combine results of two or more SELECT statements.

Operator	Description
UNION	Combines results, removes duplicates
UNION ALL	Combines results, keeps duplicates
INTERSECT	Common rows
EXCEPT / MINUS	Difference

Example:

```
SELECT name FROM table1
UNION
SELECT name FROM table2;
```

SQL Predicates -

SQL Predicates are conditions used in a WHERE, HAVING, or JOIN clause to filter records based on specific criteria. They return **TRUE, FALSE, or UNKNOWN**.

1. Comparison Predicates

Used to compare values.

Operator	Meaning
=	Equal to
<> or !=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
SELECT * FROM Customer
WHERE Age > 25;
```

2. BETWEEN Predicate

Checks if a value lies within a range (inclusive).

```
SELECT * FROM Customer
```

```
WHERE Age BETWEEN 18 AND 30;
```

Equivalent to:

```
WHERE Age >= 18 AND Age <= 30;
```

3. IN Predicate
Checks if a value matches any value in a list.

```
SELECT * FROM Customer
WHERE Country IN ('India', 'USA', 'UK');
```

4. LIKE Predicate
Used for pattern matching (mainly with strings).

Symbol	Meaning
%	Any number of characters
_	Single character

```
SELECT * FROM Customer
WHERE FirstName LIKE 'A%';
```

(Names starting with A)

5. IS NULL Predicate
Checks for NULL values.

```
SELECT * FROM Customer
WHERE Phone IS NULL;
```

6. EXISTS Predicate
Checks if a subquery returns any rows.

```
SELECT * FROM Customer C
WHERE EXISTS (
   SELECT 1 FROM Orders O
   WHERE O.CustomerID = C.CustomerID
);
```

7. ALL Predicate
Compares a value with all values returned by a subquery.

```
SELECT * FROM Customer
WHERE Age > ALL (SELECT Age FROM Customer WHERE Country = 'India');
```

8. ANY / SOME Predicate
Compares a value with any value returned by a subquery.

```
SELECT * FROM Customer
WHERE Age > ANY (SELECT Age FROM Customer WHERE Country = 'India');
```

Nested Queries-

- A **nested query** (also called a **subquery**) is a **query written inside another SQL query**.
- The result of the inner query is used by the outer query.
- The inner query executes **first**, then its result is passed to the outer query.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name operator (
   SELECT column_name
   FROM table_name
   WHERE condition
);
```

Types of Nested Queries

1. Single-Row Subquery

Returns **only one value**.

Example:
Find students older than the average age.

```
SELECT Name
FROM Student
WHERE Age > (
   SELECT AVG(Age)
   FROM Student
);
```

2. Multiple-Row Subquery

- Returns **more than one row**.
- Uses operators like **IN, ANY, ALL**.

Example:
Find students enrolled in DBMS or AI.

```
SELECT Name
FROM Student
WHERE Course IN (
   SELECT Course
   FROM Course
```

```
    WHERE Course_Name IN ('DBMS', 'AI')
);
```

3. Multiple-Column Subquery

Returns more than one column.

```
SELECT *
FROM Student
WHERE (Course, Age) IN (
    SELECT Course, Age
    FROM Alumni
);
```

4. Correlated Subquery

The inner query **depends on the outer query** and runs **once for each row**.

Example:
Find students whose age is greater than the average age of their course.

```
SELECT Name
FROM Student s
WHERE Age > (
    SELECT AVG(Age)
    FROM Student
    WHERE Course = s.Course
);
```

5. Nested Query with EXISTS

Checks whether the subquery returns any rows.

Example:

```
SELECT Name
FROM Student s
WHERE EXISTS (
    SELECT *
    FROM Enrollment e
    WHERE s.Student_ID = e.Student_ID
);
```

PL/SQL

PL/SQL Introduction

- PL/SQL (Procedural Language/SQL) is Oracle's extension of SQL that adds procedural features like loops, conditions, and error handling. It allows developers to write powerful programs that combine SQL queries with logic to control how data is processed. With PL/SQL, complex operations, calculations, and error handling can be performed directly within the Oracle database, making data manipulation more efficient and flexible.

  PL/SQL allows developers to:

  - Execute SQL queries and DML commands inside procedural blocks.
  - Define variables and perform complex calculations.
  - Create reusable program units, such as procedures, functions, and triggers.
  - Handle exceptions, ensuring the program runs smoothly even when errors occur.

Key Features of PL/SQL

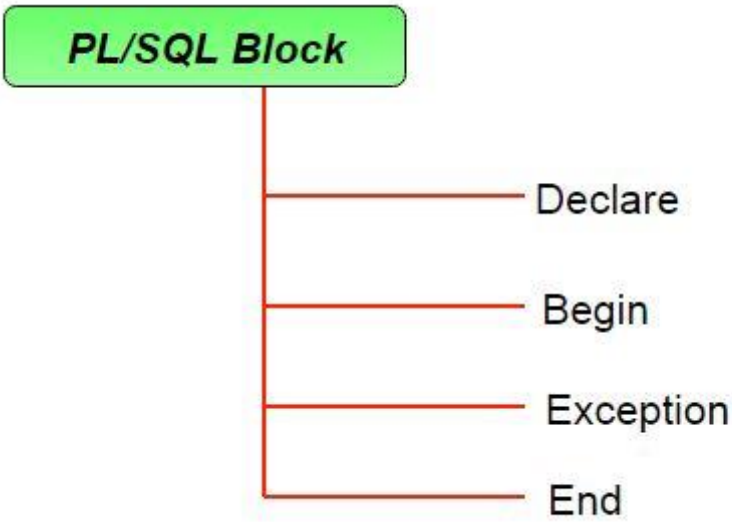
PL/SQL brings the benefits of procedural programming to the relational database world. Some of the most important features of PL/SQL include:

- **Block Structure:** PL/SQL can execute a number of queries in one block using single command.
- **Procedural Constructs:** One can create a PL/SQL unit such as procedures, functions, packages, triggers, and types, which are stored in the database for reuse by applications.
- **Error Handling:** PL/SQL provides a feature to handle the exception which occurs in PL/SQL block known as exception handling block.
- **Reusable Code**: Create stored procedures, functions, triggers, and packages, which can be executed repeatedly.
- **Performance**: Reduces network traffic by executing multiple SQL statements within a single block

Structure of PL/SQL Block

PL/SQL extends SQL by adding constructs found in **procedural languages**, resulting in a structural language that is more powerful than SQL. The basic unit in PL/SQL is a block. All PL/SQL programs are made up of blocks, which can be

nested within each other.



Typically, each block performs a logical action in the program. A block has the following structure:

DECLARE
declaration statements;

BEGIN
executable statements

EXCEPTIONS
exception handling statements

END;

PL/SQL code is written in blocks, which consist of three main sections:

- Declare section starts with **DECLARE** keyword in which variables, constants, records as cursors can be declared which stores data temporarily. It basically consists definition of PL/SQL identifiers. This part of the code is optional.
- Execution section starts with **BEGIN** and ends with **END** keyword. This is a mandatory section and here the program logic is written to perform any task like loops and conditional statements. It supports all DML commands, DDL commands and SQL*PLUS built-in functions as well.
- Exception section starts with **EXCEPTION** keyword. This section is optional which contains statements that are executed when a run-time error occurs. Any exceptions can be handled in this section.

PL/SQL Identifiers

In PL/SQL, identifiers are names used to represent various program elements like variables, constants, procedures, cursors, triggers etc. These identifiers allow you to store, manipulate, and access data throughout your PL/SQL code.

1. Variables in PL/SQL

Like several other programming languages, variables in PL/SQL must be declared prior to its use. A variable is like a container that holds data during program execution. Each variable must have a valid name and a specific **data type**.

Syntax for declaration of variables:

variable_name datatype [NOT NULL := value];

- **variable_name**: The name of the variable.
- **datatype**: The data type of the variable (e.g., INTEGER, VARCHAR2).
- **NOT NULL**: This optional constraint means the variable cannot be left empty.
- **:= value**: This optional assignment assigns an initial value to the variable.

Example: Declaring Variables

```
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE
var1 INTEGER;
var2 REAL;
var3 varchar2(20) ;

BEGIN
null;
END;
/
```

Output:

PL/SQL procedure successfully completed.

Explanation:

- **SET SERVEROUTPUT ON**: It is used to display the buffer used by the dbms_output.
- **var1 INTEGER :** It is the declaration of variable, named var1 which is of integer type. There are many other data types that can be used like float, int, real, smallint, long etc. It also supports variables used in SQL as well like NUMBER(prec, scale), varchar, varchar2 etc.
- **Slash (/) after END;:** The slash (/) tells the SQL*Plus to execute the block.
- **Assignment operator (:=)** : It is used to assign a value to a variable.

2. Displaying Output in PL/SQL

The outputs are displayed by using DBMS_OUTPUT which is a built-in package that enables the user to display output, debugging information, and send messages

from PL/SQL blocks, subprograms, packages, and triggers. Let us see an example to see how to display a message using PL/SQL :

Example: Displaying Output

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
var varchar2(40) := 'I love GeeksForGeeks' ;

BEGIN
dbms_output.put_line(var);

END;
/
```

Output:
I love GeeksForGeeks
PL/SQL procedure successfully completed.

Explanation: dbms_output.put_line : This command is used to direct the PL/SQL output to a screen.

3. Comments in PL/SQL

Like in many other programming languages, in PL/SQL also, comments can be put within the code which has no effect in the code. There are two syntaxes to create comments in PL/SQL :

- **Single Line Comment:** To create a single line comment , the symbol - - is used.
- **Multi Line Comment:** To create comments that span over several lines, the symbol /* and */ is used.

Example: Adding Comments

```
-- This is a single-line comment

/*
This is a multi-line comment
that spans over multiple lines.
*/
```

4. Taking input from users

In PL/SQL we can take input from the user and store it in a variable using **substitution variables**. These variables are preceded by an & symbol. Let us see an example to show how to take input from users in PL/SQL:

Example: Taking Input from Users

```
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE
```

```
-- taking input for variable a
a number := &amp;a;

-- taking input for variable b
b varchar2(30) := &amp;b;

BEGIN
null;

END;
/
```

Output:

Enter value for a: 24
old 2: a number := &a;
new 2: a number := 24;
Enter value for b: 'GeeksForGeeks'
old 3: b varchar2(30) := &b;
new 3: b varchar2(30) := 'GeeksForGeeks';

PL/SQL procedure successfully completed.

Explanation:

- &a and &b are substitution variables where the user will be prompted to provide values.
- The user is asked to enter values for a and b when the code runs.

PL/SQL Practical Example

Let's combine all the above concepts into one practical example. We'll create a PL/SQL block that takes two numbers from the user, calculates their sum, and displays the result.

```
--PL/SQL code to print sum of two numbers taken from the user.
SQL> SET SERVEROUTPUT ON;

SQL> DECLARE

-- taking input for variable a
a integer := &a ;

-- taking input for variable b
b integer := &b ;
c integer ;

BEGIN
c := a + b ;
dbms_output.put_line('Sum of '||a||' and '||b||' is = '||c);

END;
```

Execution:

Enter value for a: 2
Enter value for b: 3

Sum of 2 and 3 is = 5

PL/SQL procedure successfully completed.

PL/SQL Execution Environment

The PL/SQL engine resides in the **Oracle engine**. When a PL/SQL block is executed, it sends a single request to the Oracle engine, which processes the SQL and PL/SQL statements in the block together. This reduces network traffic, making PL/SQL more efficient for batch processing and handling complex logic.

Differences Between SQL and PL/SQL

Feature	SQL	PL/SQL
Purpose	SQL is a single query that is used to perform DML and DDL operations.	PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
Nature	It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
Execution	Executes single statement.	Executes block of code
Use Case	Data retrieval, manipulation and definition(eg. SELECT, INSERT, UPDATE)	Mainly used to create an application.
Syntax	SQL statements only	SQL Statements combined with procedural logic
Data Handling	Performs actions directly on the database.	Can contain SQL inside its blocks and is used for more control over data handling

PL/SQL: Stored procedures

- **PL/SQL** procedures are reusable code blocks that perform specific actions or logic within a database environment. They consist of two main components such as the procedure header which defines the procedure name & optional parameters and the procedure body which contains the executable statements implementing the desired business logic.

The procedure contains two parts:

Procedure Header

- The procedure header includes the procedure name and optional parameter list.
- It is the first part of the procedure and specifies the name and parameters

Procedure Body

- The procedure body contains the executable statements that implement the specific business logic.
- It can include declarative statements, executable statements, and exception-handling statements

Create Procedures in PL/SQL

To create a procedure in PL/SQL, use the **CREATE PROCEDURE** command:

Syntax:

```
SET ANSI_NULLS ON
SET QUOTED_IDENTIFIER ON
CREATE PROCEDURE procedure_name
@Parameter1 INT,
@Parameter2 VARCHAR(50) = NULL,
@ReturnValue INT OUTPUT
AS
BEGIN
END
GO
```

***Note**: Procedures in PL/SQL without parameters are written without parentheses after the procedure name*

Example: In this example, we will create a procedure in PL/SQL

```
SET ANSI_NULLS ON
SET QUOTED_IDENTIFIER ON
GO

CREATE PROCEDURE GetStudentDetails
    @StudentID int = 0
AS
BEGIN
    SET NOCOUNT ON;
    SELECT FirstName, LastName, BirthDate, City, Country
    FROM Students WHERE StudentID=@StudentID
END
GO
```

Explanation:

- Takes an **optional parameter** `@StudentID` (default = 0).
- **Returns**: FirstName, LastName, BirthDate, City, Country from the `Students` table.
- Uses `SET NOCOUNT ON` to suppress row count messages.
- Executes a SELECT query where `StudentID = @StudentID`.

Parameters in Procedures

In PL/SQL, parameters are used to pass values into procedures. There are three types of parameters used in procedures:

IN parameters

- Used to pass values into the procedure
- Read-only inside the procedure
- Can be a variable, literal value, or expression in the calling statement.

OUT parameters

- Used to return values from the procedure to the calling program
- Read-write inside the procedure
- Must be a variable in the calling statement to hold the returned value

IN OUT parameters

- Used for both passing values into and returning values from the procedure
- Read-write inside the procedure
- Must be a variable in the calling statement

Modify Procedures in PL/SQL

To modify an existing procedures in PL/SQL use the **ALTER PROCEDURE command**:

Syntax

```
ALTER PROCEDURE Syntax is:
SET ANSI_NULLS ON
SET QUOTED_IDENTIFIER ON
GO
ALTER PROCEDURE procedure_name
@Parameter1 INT,
@Parameter2 VARCHAR(50) = NULL,
@ReturnValue INT OUTPUT
AS
BEGIN
-- Query
END
GO
```

Example: In this example, we will modify a procedure in PL/SQL:

```
SET ANSI_NULLS ON
SET QUOTED_IDENTIFIER ON
GO

ALTER PROCEDURE GetStudentDetails
    @StudentID int = 0
AS
BEGIN
    SET NOCOUNT ON;
    SELECT FirstName, LastName, City
    FROM Students WHERE StudentID=@StudentID
END
GO
```

Explanation: This code updates the `GetStudentDetails` procedure to:

- Take `@StudentID` as input (default = 0)
- Return only `FirstName`, `LastName`, and `City` from the `Students` table
- Suppress row count message using `SET NOCOUNT ON`

Drop Procedure in PL/SQL

To drop a procedure in PL/SQL use the **DROP PROCEDURE** command

Syntax:

```
DROP PROCEDURE procedure_name
```

PL/SQL DROP PROCEDURE Example

In this example, we will delete a procedure in PL/SQL

DROP PROCEDURE GetStudentDetails

Advantages of Procedures

- They result in performance improvement of the application. If a procedure is being called frequently in an application in a single connection, then the compiled version of the procedure is delivered.
- They reduce the traffic between the database and the application since the lengthy statements are already fed into the database and need not be sent again and again via the application.
- They add to code reusability, similar to how functions and methods work in other languages such as **C**/**C++** and **Java**.

Disadvantages of Procedures

- Stored procedures can cause a lot of memory usage. The database administrator should decide an upper bound as to how many stored procedures are feasible for a particular application.
- **MySQL** does not provide the functionality of debugging the stored procedures.

PL/SQL: functions

PL/SQL functions are reusable blocks of code that can be used to perform specific tasks. They are similar to **procedures** but must always return a value.

A function in PL/SQL contains:

- **Function Header**: The function header includes the function name and an optional parameter list. It is the first part of the function and specifies the name and parameters.
- **Function Body**: The function body contains the executable statements that implement the specific logic. It can include declarative statements, executable statements, and exception-handling statements.

Create Function in PL/SQL

To create a procedure in PL/SQL, use the **CREATE FUNCTION statement.**

Syntax

The syntax to create a function in PL/SQL is given below:

```
CREATE [OR REPLACE] FUNCTION function_name
(parameter_name type [, ...])

-- This statement is must for functions
RETURN return_datatype

{IS | AS}

BEGIN
-- program code

[EXCEPTION
exception_section;

END [function_name];
```

Example

In this example, we create a PL/SQL function to calculate factorial of a number

```
CREATE OR REPLACE FUNCTION factorial(x NUMBER)
  RETURN NUMBER
IS
  f NUMBER;
BEGIN
  IF x = 0 THEN
    f := 1;
  ELSE
    f := x * factorial(x - 1);
  END IF;
  RETURN f;
END;
```

How to Call Function in PL/SQL

To call a function, specify the function name and any required parameters. The function will execute and return a value.

Example

Here, we call the factorial function which we created earlier.

```
DECLARE
  num NUMBER;
  result NUMBER;
BEGIN
  num := 5;
```

```
  result := factorial(num);
  DBMS_OUTPUT.PUT_LINE('Factorial of ' || num || ' is ' || result);
END;
```

Output:

```
the reverse of number is  987654321
```

PL/SQL Recursive Function

A PL/SQL recursive function is a function that calls itself to perform a specific task. The function continues to call itself until a certain condition is met, at which point it returns a value.

Recursive Function Example

Lets implement a recursive function to calculate the factorial of a number **Recursive functions example:**

```
DECLARE
  num INT;
  answer INT;

  -- Defining the function
  FUNCTION factorial(x NUMBER)
    RETURN INT
  IS
    f INT;
  BEGIN
    IF x = 0 THEN
      f := 1;
    ELSE
      f := x * factorial(x - 1);
    END IF;
    RETURN f;
  END;

BEGIN
  num := 5;
  answer := factorial(num);
  DBMS_OUTPUT.PUT_LINE('Factorial of ' || num || ' is ' || answer);
END;
```

Output:

```
 Factorial of  5 is 120
```

Exception handling in PL/SQL Functions

Exception handling can be done using an exception block in functions but exception handling using a *try-catch* block cannot be done.

Example:

```
SET SERVEROUTPUT ON;

DECLARE
  a INT;
  b FLOAT;
  myexp EXCEPTION;

  FUNCTION sqroot(x INT)
    RETURN FLOAT
  AS
    answer FLOAT;
  BEGIN
    IF x < 0 THEN
      RAISE myexp;
    ELSE
      answer := SQRT(x);
    END IF;
    RETURN answer;
  EXCEPTION
    WHEN myexp THEN
      DBMS_OUTPUT.PUT_LINE('Square root of a negative number is not allowed,
so returning the same number');
      RETURN x;
  END;

BEGIN
  b := sqroot(-2);
  DBMS_OUTPUT.PUT_LINE('The value is ' || b);
END;
```

Output:

```
square of negative number is  not allowed so returning the same number
the value is -2
```

Advantages of PL/SQL Functions

1. We can make a single call to the database to run a block of statements thus it improves the performance against running SQL multiple times. This will reduce the number of calls between the database and the application.
2. We can divide the overall work into small modules which becomes quite manageable also enhancing the readability of the code.
3. It promotes reusability.
4. It is secure since the code stays inside the database thus hiding internal database details from the application(user). The user only makes a call to the PL/SQL functions. Hence security and data hiding is ensured.

DROP Function in PL/SQL

To drop a function in PL/SQL, **DROP function statement** is used.

Syntax

DROP Function <function_name>;

Example

DROP Function func1;

PL/SQL: triggers

PL/SQL triggers are block structures and predefined programs **invoked automatically** when some event occurs. They are stored in the database and **invoked repeatedly** in a particular scenario. There are two states of the triggers, they are **enabled** and **disabled**. When the trigger is created it is enabled. **CREATE TRIGGER** statement creates a trigger. A triggering event is specified on a table, a view, a schema, or a database.**BEFORE** and **AFTER** are the trigger Timing points.**DML triggers** are created on a table or view, and **triggers.** Crossedition triggers are created on Edition-based redefinition. System Triggers are created on schema or database using DDL or database operation statements.It is applied on new data only ,it don't affect existing data.

They are associated with response-based events such as a

- **Database Definition Language** statements such as CREATE, DROP or ALTER.
- **Database Manipulation Language** statements such as UPDATE, INSERT or DELETE.
- Database operations such as LOGON, LOGOFF, STARTUP, and SHUTDOWN .

Why are Triggers important?

The importance of Triggers are:

- **Automated Action:** It helps to automate actions in response to events on table or views.
- **Data integrity**: Constraint can be applied to the data with the help of trigger.It is used to ensure
- **Consistency**: It helps to maintain the consistency of the database by performing immediate responses to specific events.

- **Error handling**: It helps in error handling by responding to the errors. For example, If specific condition is not met it will provide an error message.

PL/SQL Trigger Structure

Triggers are fired on the tables or views which are in the database. Either table, view ,schema, or a database are the basic requirement to execute a trigger. The trigger is specified first and then the action statement are specified later.

Syntax:

```
CREATE OR REPLACE TRIGGER trigger_name
BEFORE or AFTER or INSTEAD OF //trigger timings
INSERT or UPDATE or DELETE // Operation to be performed
of column_name
on Table_name
FOR EACH ROW
DECLARE
        Declaration section
BEGIN
        Execution section
EXCEPTION
        Exception section
END;
/
```

Query operation to be performed i.e INSERT,DELETE,UPDATE.

- CREATE [OR REPLACE] TRIGGER trigger_name is used to create a trigger or replace the existing trigger.|
- BEFORE | AFTER | INSTEAD OF specifies trigger timing.
- INSERT | UPDATE | DELETE are the DML operations performed on table or views.
- OF column_name specifies the column that would be updated.
- ON table_name species the table for the operation.
- FOR EACH ROW specify that trigger is executed on each row .

Types of PL/SQL Triggers

Trigger timing and operations forms different combinations such as BEFORE INSERT OR BEFORE DELETE OR BEFORE UPDATE .BEFORE and AFTER are known as conditional triggers.

Conditional Trigger: Before

Trigger is activated before the operation on the table or view is performed.

Query:

```
-- Create Geeks table
CREATE TABLE Geeks (
    Id INT,
    Name VARCHAR2(20),
    Score INT
);

-- Insert into Geeks Table
INSERT INTO Geeks (Id, Name, Score) VALUES (1, 'Sam', 800);
INSERT INTO Geeks (Id, Name, Score) VALUES (2, 'Ram', 699);
INSERT INTO Geeks (Id, Name, Score) VALUES (3, 'Tom', 250);
INSERT INTO Geeks (Id, Name, Score) VALUES (4, 'Om', 350);
INSERT INTO Geeks (Id, Name, Score) VALUES (5, 'Jay', 750);
-- insert statement should be written for each entry in Oracle Sql Developer

CREATE TABLE Affect (
    Id INT,
    Name VARCHAR2(20),
    Score INT
);

-- BEFORE INSERT trigger
CREATE OR REPLACE TRIGGER BEFORE_INSERT
BEFORE INSERT ON Geeks
FOR EACH ROW
BEGIN
    INSERT INTO Affect (Id, Name, Score)
    VALUES (:NEW.Id, :NEW.Name, :NEW.Score);
END;
/
INSERT INTO Geeks (Id, Name, Score) VALUES (6, 'Arjun', 500);
```

BEFORE DELETE Trigger

```
-- BEFORE DELETE trigger
CREATE OR REPLACE TRIGGER BEFORE_DELETE
BEFORE DELETE ON Geeks
FOR EACH ROW
BEGIN
    INSERT INTO Affect (Id, Name, Score)
    VALUES (:OLD.Id, :OLD.Name, :OLD.Score);
END;
/
DELETE FROM Geeks WHERE Id = 3;
```

BEFORE UPDATE Trigger

```
-- BEFORE UPDATE trigger
CREATE OR REPLACE TRIGGER BEFORE_UPDATE
BEFORE UPDATE ON Geeks
FOR EACH ROW
```

```
BEGIN
   INSERT INTO Affect (Id, Name, Score)
   VALUES (:OLD.Id, :OLD.Name, :OLD.Score);
END;
/
UPDATE Geeks SET Score = 900 WHERE Id = 5;
SELECT * FROM Affect;
SELECT * FROM Geeks;
```

Output:

```
Task completed in 0.184 seconds

Trigger BEFORE_INSERT compiled

1 row inserted.

Trigger BEFORE_DELETE compiled

1 row deleted.

Trigger BEFORE_UPDATE compiled

1 row updated.

        ID NAME                      SCORE
---------- -------------------- ----------
         6 Arjun                       500
         3 Tom                         250
         5 Jay                         750
```

Conditional Trigger Before

Explanation:

- BEFORE_INSERT Trigger is fired before adding a row in Geeks Table,and row is inserted in the Affect table.
- BEFORE_DELETE Trigger is activated before the row is delete from the Geeks table and row which satisfy the condition is added to Affect table.

- BEFORE_UPDATE TRIGGER is activated before the row with Id=5 is updated and row with old values is added to Affect table

Geeks table after trigger events

	ID	NAME	SCORE
1	1	Sam	800
2	2	Ram	699
3	4	Om	350
4	5	Jay	900
5	6	Arjun	500

Conditional Trigger Before

Conditional Trigger: After

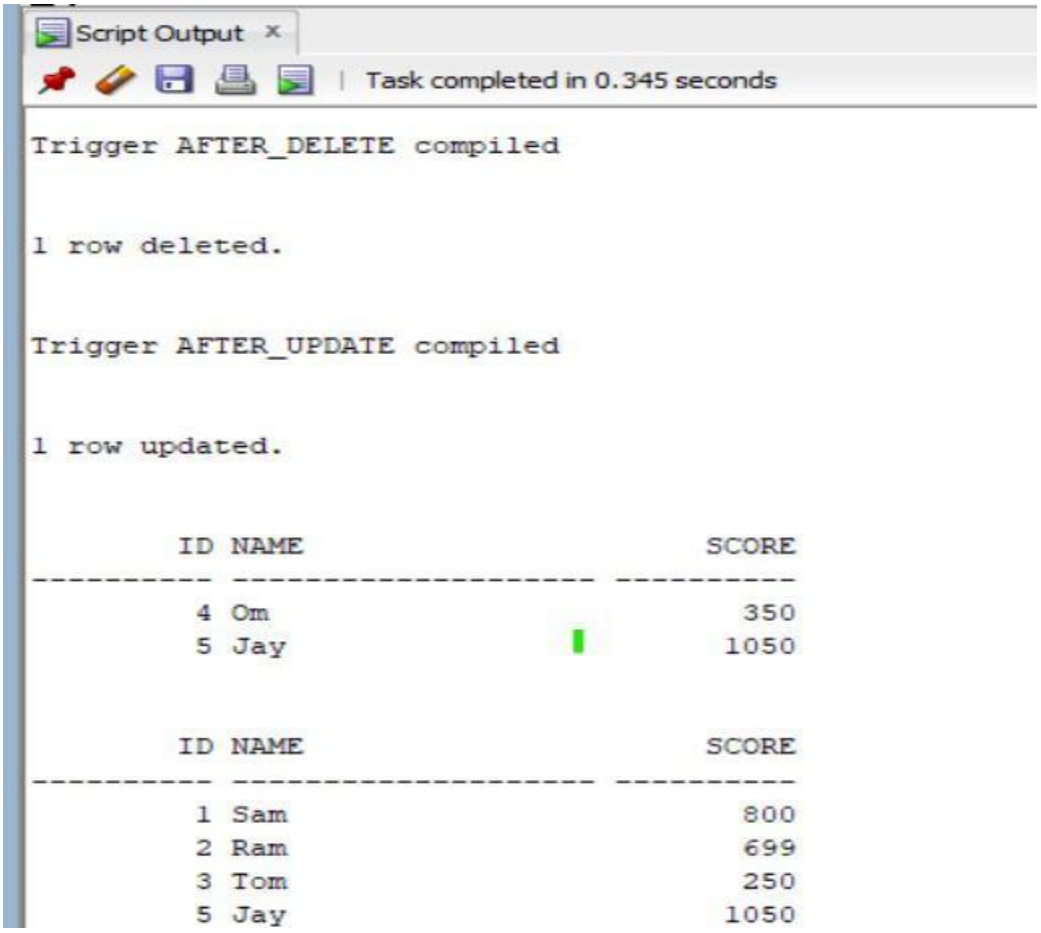
Trigger is activated after the operation on the table or view is performed.

Query:

```
SET SERVEROUTPUT ON;


CREATE TABLE Geeks (
   Id INT,
   Name VARCHAR2(20),
   Score INT
);


-- Insert into Geeks Table
INSERT INTO Geeks (Id, Name, Score) VALUES (1, 'Sam', 800);
INSERT INTO Geeks (Id, Name, Score) VALUES (2, 'Ram', 699);
INSERT INTO Geeks (Id, Name, Score) VALUES (3, 'Tom', 250);
INSERT INTO Geeks (Id, Name, Score) VALUES (4, 'Om', 350);
INSERT INTO Geeks (Id, Name, Score) VALUES (5, 'Jay', 750);
-- insert statement should be written for each entry in Oracle Sql Developer


CREATE TABLE Affect (
   Id INT,
   Name VARCHAR2(20),
   Score INT
);
SELECT * FROM Geeks;
-- AFTER DELETE trigger
CREATE OR REPLACE TRIGGER AFTER_DELETE
AFTER DELETE ON Geeks
FOR EACH ROW
```

```
BEGIN
   INSERT INTO Affect (Id, Name, Score)
   VALUES (:OLD.Id, :OLD.Name, :OLD.Score);
END;
/
DELETE FROM Geeks WHERE Id = 4;


-- AFTER UPDATE trigger
CREATE OR REPLACE TRIGGER AFTER_UPDATE
AFTER UPDATE ON Geeks
FOR EACH ROW
BEGIN
   INSERT INTO Affect (Id, Name, Score)
   VALUES (:NEW.Id, :NEW.Name, :NEW.Score);
END;
/
UPDATE Geeks SET Score = 1050 WHERE Id = 5;
SELECT * FROM Affect;
SELECT * FROM Geeks;
```

Output:



Conditional Trigger After

Explanation: After the deletion of the row from the Geek table trigger is fired and the row which is deleted is added to the Affect Table.In second trigger i.e After_update trigger is fired after performing update on Geeks table and the row is added to Affect Table.Output contains the Affect table and the Geek table after the trigger events.

PL/SQL: assertions

- An assertion is a check placed in your code to ensure that a condition is true at a specific point during execution.
- If the condition is false, the program raises an error and stops (or handles it).

Why use Assertions?

- To validate assumptions in code
- To catch logical errors early
- To make programs more reliable and readable
- Very useful during debugging and testing

1. Assertion using IF + RAISE_APPLICATION_ERROR

- This is the most common method.

Example:

```
DECLARE
  v_age NUMBER := 15;
BEGIN
   IF v_age < 18 THEN
       RAISE_APPLICATION_ERROR(-20001, 'Age must be 18 or above');
   END IF;
   DBMS_OUTPUT.PUT_LINE('Eligible for voting');
END;
/
```

2. Assertion in Procedure

Example:

```
CREATE OR REPLACE PROCEDURE withdraw_amount (
    p_balance NUMBER,
    p_amount  NUMBER
) IS
```

```
BEGIN
   IF p_amount > p_balance THEN
      RAISE_APPLICATION_ERROR(-20002, 'Insufficient balance');
   END IF;
   DBMS_OUTPUT.PUT_LINE('Withdrawal successful');
END;
/
```

Assertions	Triggers
We can use Assertions when we know that the given particular condition is always true.	We can use Triggers even particular condition may or may not be true.
When the SQL condition is not met then there are chances to an entire table or even Database to get locked up.	Triggers can catch errors if the condition of the query is not true.
Assertions are not linked to specific table or event. It performs task specified or defined by the user.	It helps in maintaining the integrity constraints in the database tables, especially when the primary key and foreign key constraint are not defined.
Assertions do not maintain any track of changes made in table.	Triggers maintain track of all changes occurred in table.
Assertions have small syntax compared to Triggers.	They have large Syntax to indicate each and every specific of the created trigger.
Modern databases do not use Assertions.	Triggers are very well used in modern databases.
Purpose of assertions is to Enforces business rules and constraints.	Purpose of triggers is to Executes actions in response to data changes.
Activation is checked after a transaction completes	Activation is activated by data changes during a transaction

Assertions	Triggers
Granularity applies to the entire database	Granularity applies to a specific table or view
Syntax Uses SQL statements	Syntax Uses procedural code (e.g. PL/SQL, T-SQL)
Error handling Causes transaction to be rolled back.	Error handling can ignore errors or handle them explicitly
Assertions may slow down performance of queries.	Triggers Can impact performance of data changes.
Assertions are Easy to debug with SQL statements.	Triggers are more difficult to debug procedural code
Examples- CHECK constraints, FOREIGN KEY constraints	Examples - AFTER INSERT triggers, INSTEAD OF triggers

PL/SQL: roles

- A role is a named group of privileges that can be granted to users or other roles to simplify privilege management.
- Instead of granting many privileges one by one, you grant one role.

Types of Privileges in Roles

System privileges
(CREATE TABLE, CREATE VIEW, CREATE PROCEDURE, etc.)

Object privileges
(SELECT, INSERT, UPDATE, DELETE on tables, views, etc.)

Creating a Role

```
CREATE ROLE manager_role;
```

Granting Privileges to a Role

System Privilege:

```
GRANT CREATE TABLE TO manager_role;
```

Object Privilege:

```
GRANT SELECT, INSERT ON employee TO manager_role;
```

Enabling / Disabling Roles

```
GRANT manager_role TO scott;
```

Revoking Roles

```
REVOKE manager_role FROM scott;
```

PL/SQL: privileges

- **Privileges** are **permissions** granted to a user to **perform specific actions** on the database or database objects.
- They control **who can do what** in Oracle.

Types of Privileges in PL/SQL

1. System Privileges
Allow users to perform **database-level actions**.
Grant System Privilege:

```
GRANT CREATE TABLE TO scott;
```

2. Object Privileges
Allow users to perform actions on **specific database objects**.

Grant Object Privilege:

```
GRANT SELECT, INSERT ON employee TO scott;
```

Granting Privileges
Allows the user to **pass the privilege to others**.

```
GRANT SELECT ON employee TO scott WITH GRANT OPTION;
```

Revoking Privileges

```
REVOKE SELECT ON employee FROM scott;
```